LabVIEW 기본문법

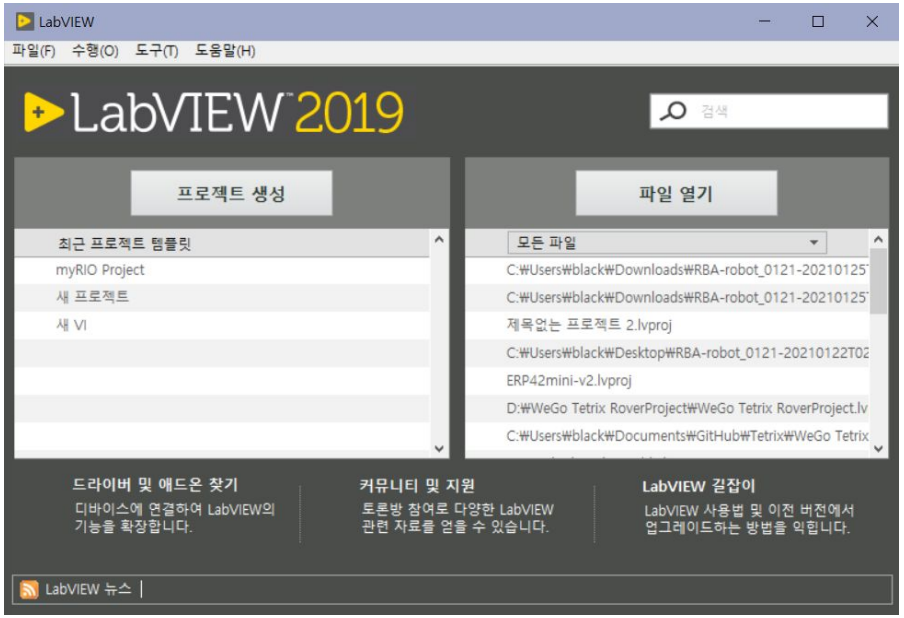
LabVIEW 기본문법

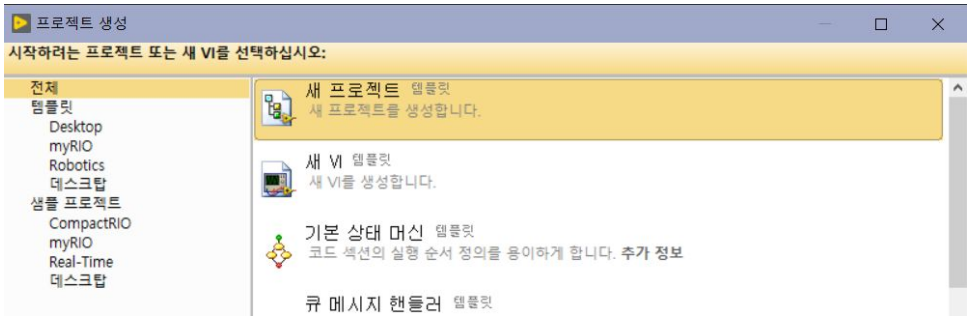
1. LabVIEW 시작하기
2. LabVIEW 데이터 타입
   I. 숫자형 데이터
   II. 불리언 데이터
   III. 문자열 데이터
3. LabVIEW 구조
   I. While 루프
   II. For 루프
   III. 루프의 입출력
   IV. 시프트레지스터
   V. 케이스 구조
   VI. 이벤트 구조
4. 타이밍 사용하기

(추가)

5. 배열 사용하기
   I. 배열
   II. 배열 함수
6. 클러스터 사용하기
   I. 클러스터
   II. 그래프와 차트
7. 변수 사용하기
   I. 로컬 변수
8. Sub VI

1. LabVIEW 시작하기



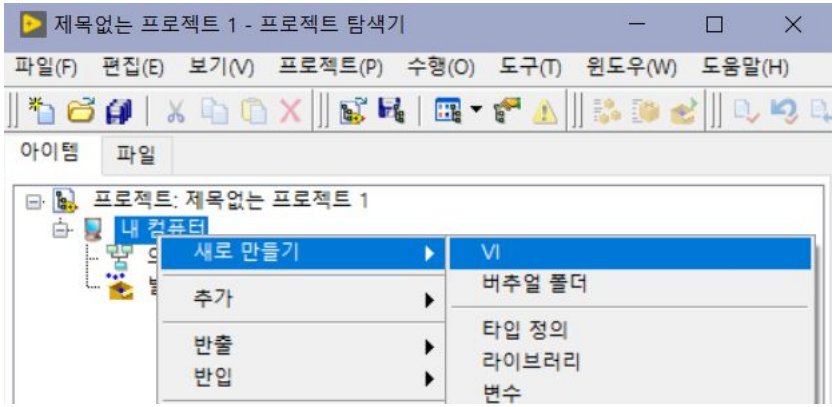


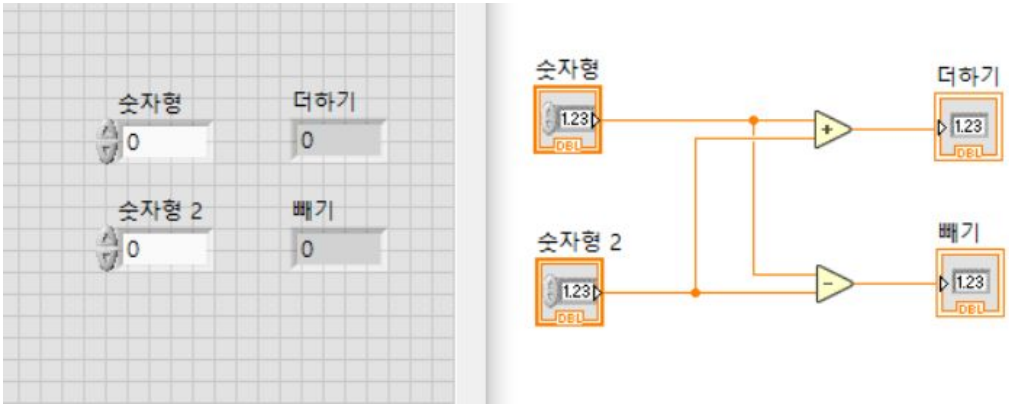
프로젝트 생성 : 새로운 랩뷰 프로젝트를 생성한다.

파일 열기 : 기존에 저장 된 랩뷰 파일을 연다.

프로젝트 생성 버튼을 누른 다음, 새 프로젝트를 클릭한다.

용도에 맞춰서, 사용하는 하드웨어에 맞춰서 프로젝트를 구성할 수 있으나, 현재는 기본기능만 탑재한 프로젝트를 생성한다.





내 컴퓨터 – 새로 만들기 – VI로 프로그램 환경을 민든다. 이제 여기서 프로그래밍을 할 수 있다.
Crrl+T로 화면을 정리하자

좌측은 콘솔창을 겸하는 프런트패널, 우측은 프로그래밍을 하는 블록다이어그램이다.
연결이 불량일 시, 와이어가 깨지니 주의하자.

위 예제에서 값이 입력되는 부분을 컨트롤, 값이 출력되는 부분을 인디케이터라고 한다.

2. LabVIEW 데이터 타입

부동소수

- 정수부분과 소수부분을 가변적으로 처리해주는 숫자형
- 32bit SGL, 64bit DBL, 128bit EXT
- 개발에 편리하지만 용량관리에 불리하다.

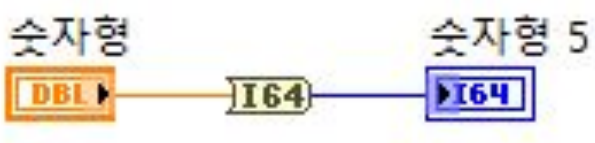
고정소수

- FXP 정수부분과 소수부분을 지정할 수 있는 숫자형
- 용량 관리에 유리하나 설정한 데이터 타입에 따라서 오차가 있을 수가 있다.

정수

- 부호있음과 부호 없음으로 나뉜다.
- 정수부분이 있으나 소수부분이 없다.

2. LabVIEW 데이터 타입



강제형변환

- 동일한 형끼리의 와이어만 연결가능
- 다른 형끼리의 경우 프로그램이 스스로 변경시켜 강제로 연결시킨다.
- 시스템 안정성을 저해하기에 어플리케이션을 다운시키는 경우도 존재한다.
- 따라서 실전 어플리케이션에서는 절대 사용하지 않는다.

- 반드시 변환 팔레트를 사용할 것.

숫자형 함수 팔레트

더하기, 빼기, 곱하기, 나누기 : 사칙연산

몫&나머지 : 나누기를 하고 몫과 나머지를 추출한다.

변환 : 형을 변환하는 함수

증가, 감소 : 1을 더하거나 뺀다.

배열원소 더하기/곱하기 : 배열에 있는 모든 값을 더하거나 곱한다.

복합연산 : 반복 연산 함수

2의 거듭 제곱 : Ax2^n를 계산한다. 나눗셈을 대체할 수 있다.

부호 : 입력이 양수라면 1, 음수라면 -1, 0이라면 0을 반환한다.

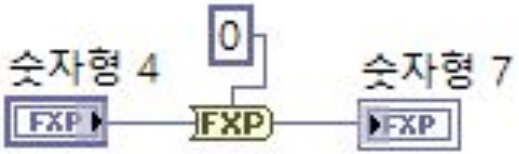
난수 : 0~1 사이의 난수를 출력한다.

절대값, 반올림, 버림, 올림, 제곱근, 제곱, 음수화, 역수는 이름 그대로 이므로 설명 생략

FXP : 고정 소수점 실습

고정 소수점은 프로퍼티로 데이터 타입을 수정할 수 있다.

그러나 같은 FXP라도 데이터 타입이 다르면 강제형변환이 일어난다



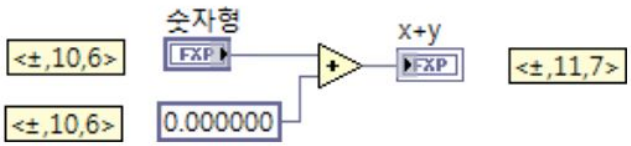
서로 다른 강제형변환을 연결해보자.

- 입력, 출력 서로 다른 타입의 FXP를 연결한다.
- 출력부를 우 클릭해서 상수를 만든다.
- 변환 팔레트 위쪽에 연결한다.

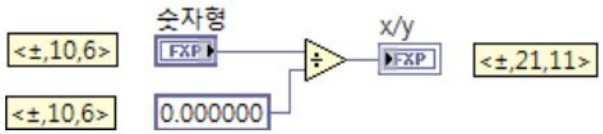
FXP : 고정 소수점 실습

고정 소수점으로 연산을 하면 다음과 데이터 형에 다음과 같은 변형이 일어난다.

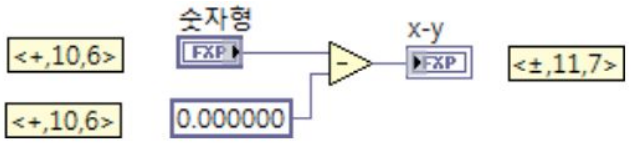
- 실제로 해보고 확인해봅시다.



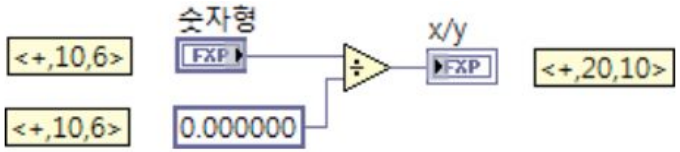
덧셈 : 정수부분+1, 소수부분+1



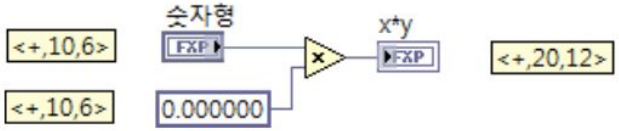
나눗셈(부호있음) : 정수부분1+정수부분2+1, 소수부분1+정수부분2-소수부분2+1



뺄셈 : 정수부분+1, 소수부분+1



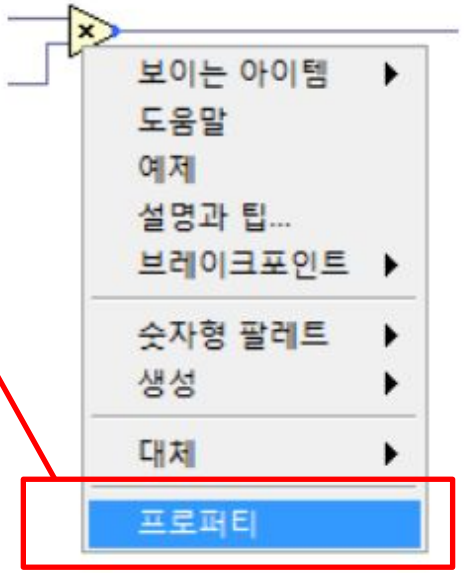
나눗셈(부호없음) : 정수부분1+정수부분2, 소수부분1+정수부분2-소수부분2



곱셈 : 정수부분1+정수부분2, 소수부분1+소수부분2

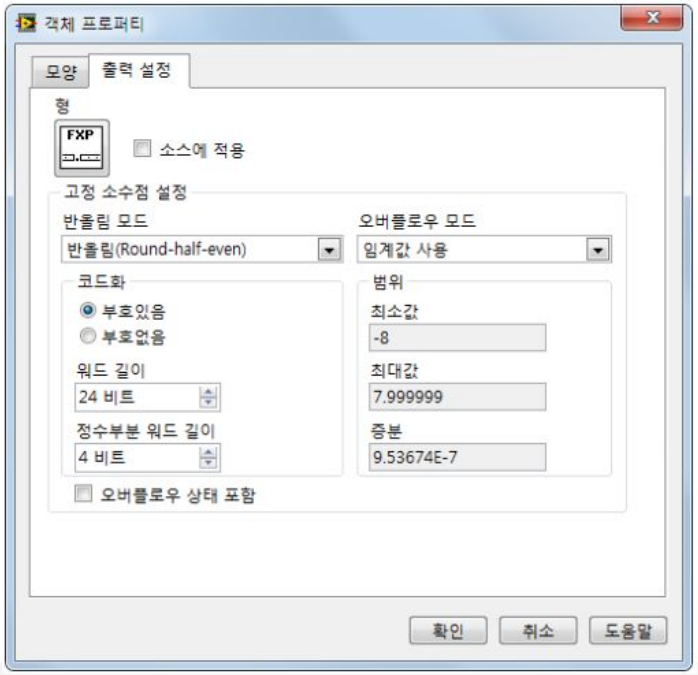
고정 소수점은 출력 설정이 가능하다.

(!) 상수가 아닌 함수의 출력 설정



- 관련된 모든 함수에 같은 형식 사용
- 여러 고정 소수점 설정 타입을 혼합하면 데이터 손실이 발생할 수 있음
- 수학 함수가 필요에 따라 임계값 사용, 잘라내기, 반올림을 수행하도록 설정

FXP의 프로퍼티와 약간 다르니 주의



반올림 모드

- 잘라버림
- 반올림(Round Half-Up) – 우리가 흔히 쓰는 거
- 반올림(Round Half-Even) (기본) – x.5가 걸릴시, 가장 가까운 짝수로 보낸다.

오버플로우

- 유효자리 줄이기 : 값 초과시 오버플로우
- 임계값 사용 : 오버플로우 LED만 점등

$$x' = \frac{x}{\sqrt{x^2+y^2+z^2}} \quad y' = \frac{y}{\sqrt{x^2+y^2+z^2}} \quad z' = \frac{z}{\sqrt{x^2+y^2+z^2}}$$

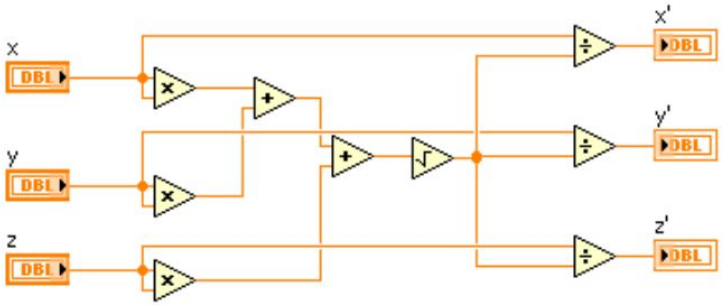
고정 소수점 알고리즘의 개발과정

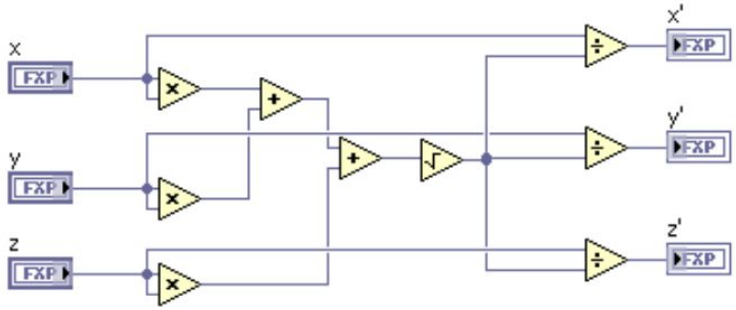
1. 기호형

1. 부동 소수점 데이터 타입을 사용하여 알고리즘 테스트

1. 고정 소수점 데이터 타입을 이용하여 용량을 최대한 줄이며 정확도가 허용 범위 내인지 확인

1. FPGA 컴파일 테스트

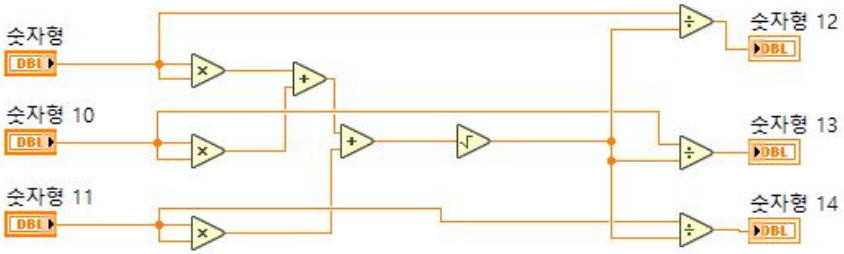


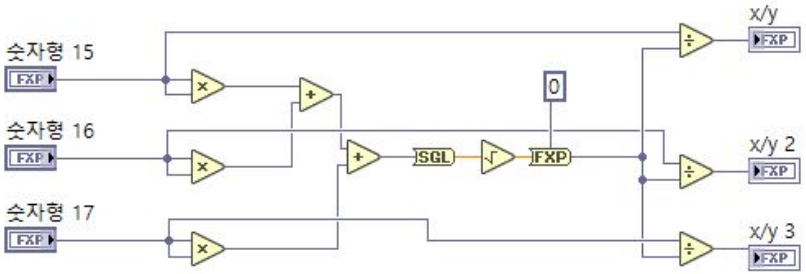


실습 2 : 다음을 만들어보자.

$$x' = \frac{x}{\sqrt{x^2+y^2+z^2}} \quad y' = \frac{y}{\sqrt{x^2+y^2+z^2}} \quad z' = \frac{z}{\sqrt{x^2+y^2+z^2}}$$

- X = 정수부분 15비트, 소수부분 10비트, 부호있음
- Y = 정수부분 12비트, 소수부분 8비트, 부호있음
- Z = 정수부분 10비트, 소수부분 7비트, 부호있음
- 모든 함수들은 반올림(Round Half-Even) 적용
- 제곱근은 SGL, FXP 변환 팔레트를 사용하고, 그 사이에 정수부분 14비트, 소수부분 7비트, 부호있음으로 구성 된 상수를 넣으십시오.



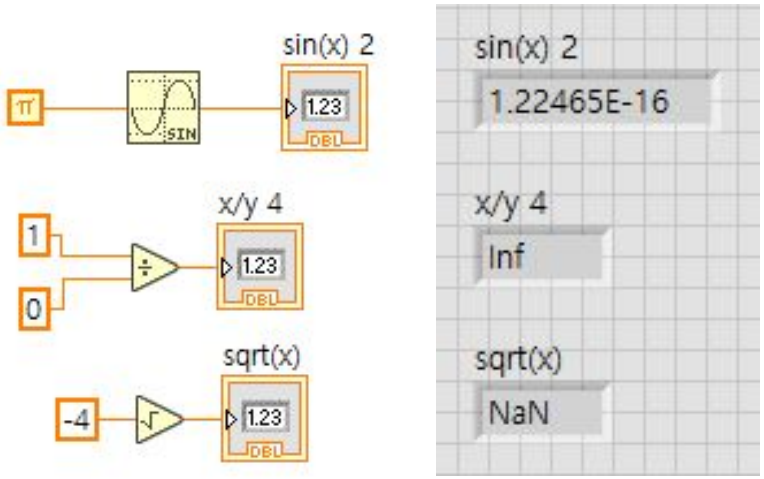


- X', Y', Z'의 크기를 확인하십시오,

- 부동 소수점과 고정소수점의 오차를 확인하십시오.
- 필요한 결과값의 소수부분이 2자리 뿐이라면 어떻게 바꾸시겠습니까?

-> 오버플로우로 인한 값 오류 주의

부동 소수 연산

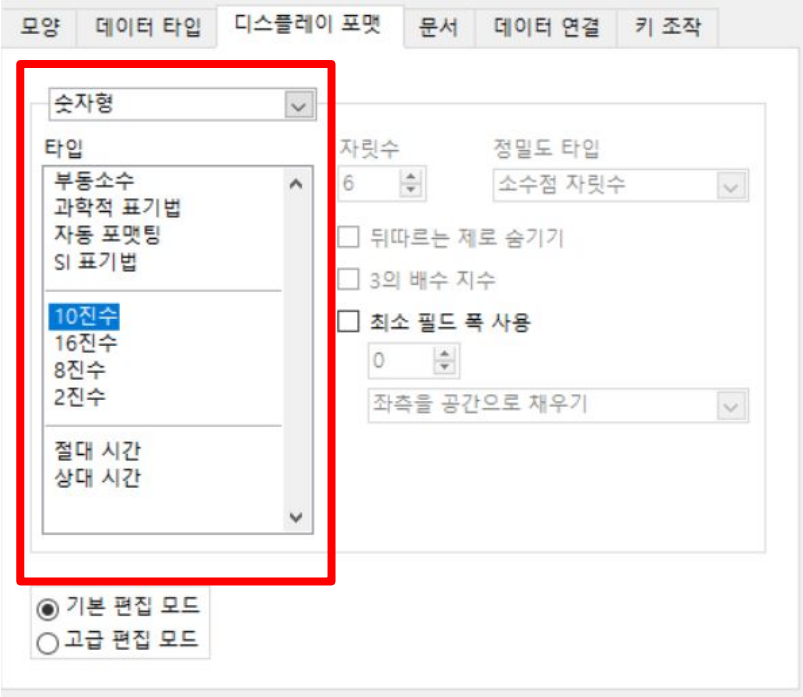


1. 삼각함수는 라디안을 입력하고 삼각함수를 실수형으로 출력합니다.(결과 값은 √3/2)
2. 0으로 나누는 경우 연산 불가 = 무한이 출력
3. 허근의 경우 NaN이 출력된다. -> 별도의 허수 함수 사용 필요

슬라이드

슬라이드는 숫자형을 그래픽으로 사용할 수 있도록 만든 컨트롤, 인디케이터이다.
디지털 디스플레이로 구체적인 숫자를 볼 수 있다.
우클릭 – 슬라이더 추가를 통해 여러 개의 숫자를 입력할 수 있다.

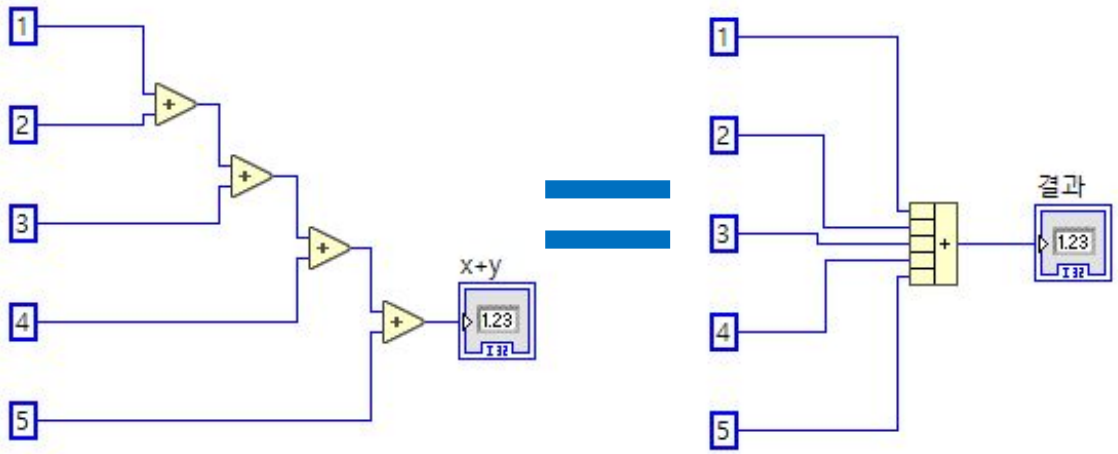
디스플레이 포맷, 기수

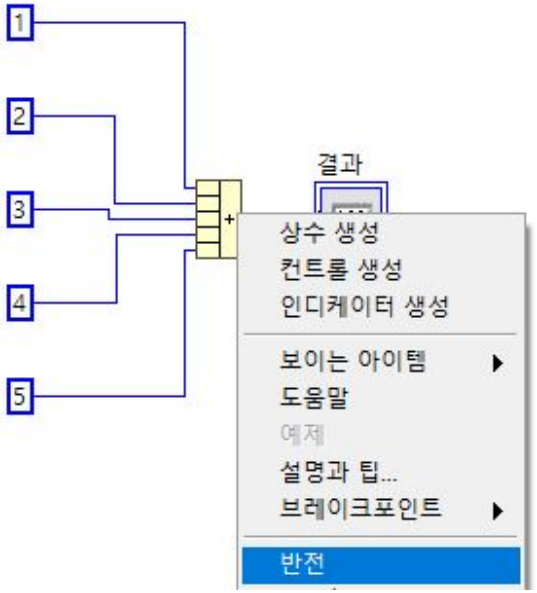


숫자형의 프로퍼티에서 숫자의 출력 방식과 기수를 변경할 수 있습니다.

프로퍼티에 들어가서 직접 조작해보며 관찰해봅시다.

복합 연산

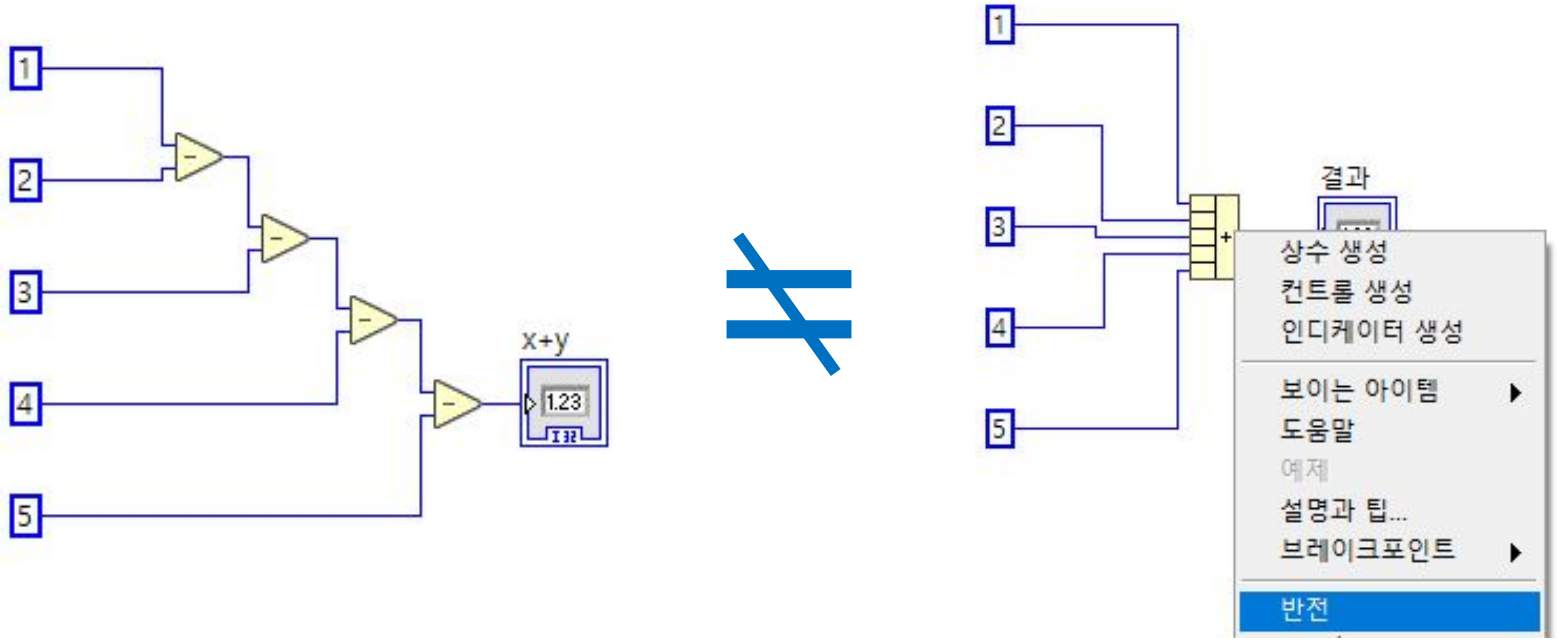




복합 연산은 반복 연산을 위해 사용하는 함수이며, 위의 결과값은 모두 15로 같다.

복합 연산은 더하기 곱하기, 논리 연산을 지원한다.
앞 뒤로 반전을 추가할 수 있는데, 반전을 추가하면 더하기는 음수가 된다.

복합 연산

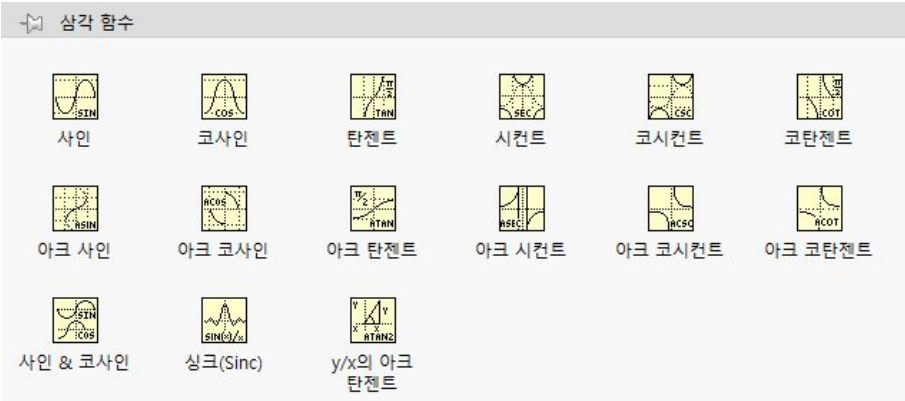


그러나 반전이 추가 된 것의 부호가 바뀌는 것이므로 위의 경우에는

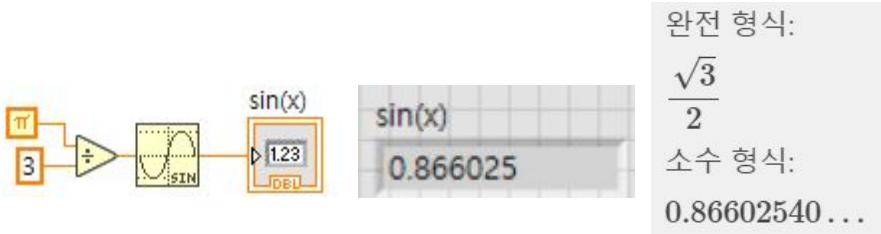
왼쪽 : 1-2-3-4-5=-13

오른쪽 : -(1+2+3+4+5)=-15가 되므로 주의해야한다.

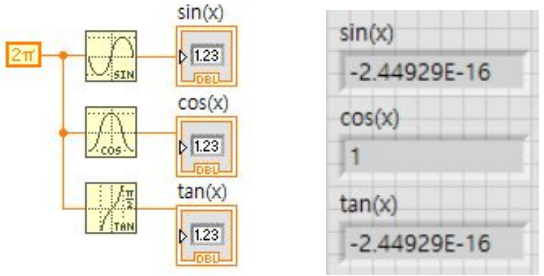
삼각 함수



블록다이어 그램 우클릭 – 수학 – 기본 & 특수 함수 - 삼각함수



기본적으로 입출력은 각도가 아닌 라디안으로 이루어진다.
다만, 출력의 경우 DBL 숫자형을 이용하여 계산하므로 소수 형식으로 나온다



0 대신 근접한 작은 수

0 대신 근접한 작은 수

또한 DBL 숫자형을 이용하여 계산하므로 정확히 0이 나오지 않기도 한다.

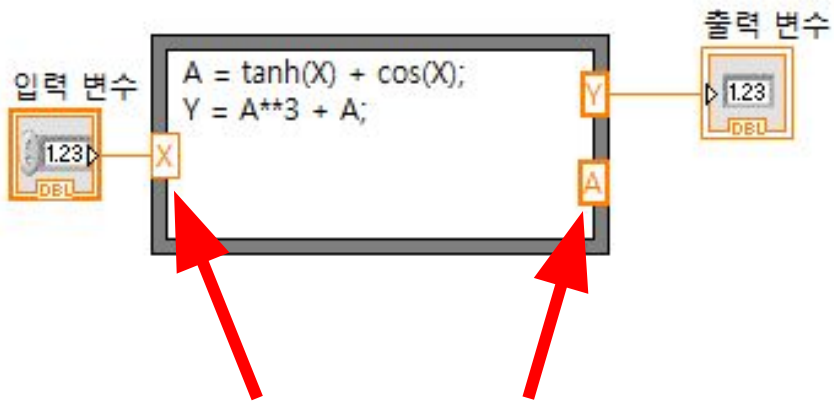
LabVIEW에서는 삼각함수 연산이 이론적인 수학 값과 완벽하게 일치하지 않으므로 주의해야한다.

수식 노드

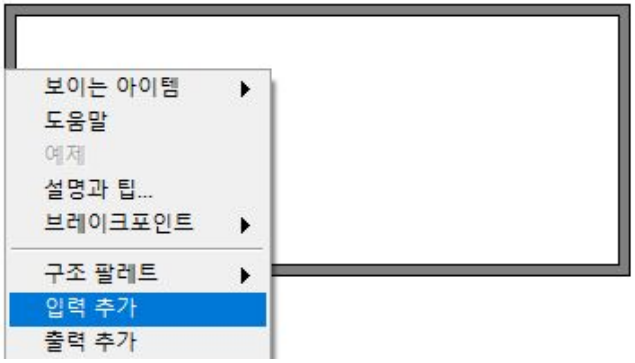
수식 노드는 C 프로그래밍과 유사하게 텍스트 기반의 구문으로 수학 연산을 시행 할 수 있습니다.

또한 if, While, for, Do 루프 등을 사용할 수 있습니다.

C프로그래밍과 비슷하지만 완전히 같지 않으므로 주의합시다.



C 언어처럼 int a;와 같이 선언할 필요는 없지만,
대신 입출력 터널을 만들어 줘야합니다.



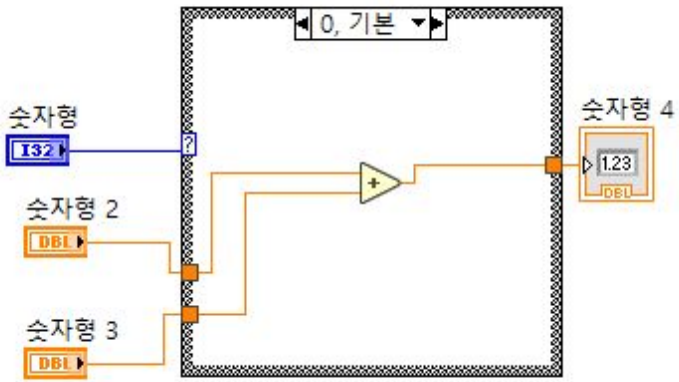
입출력 터널은 수식 노드를 우클릭 해서 생성합니다.

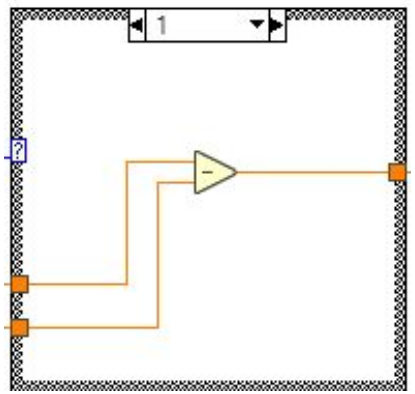
(!) LabVIEW에서는 if 문 구조가 없기 때문에 자주 사용된다.

링, 열거형

링, 열거형에 대해 설명하기 전, 잠시 케이스 문에 대해 설명하도록 하겠습니다.

케이스 문은 함수-구조에 있으며 입력 된 변수에 따라서 다른 동작을 수행 할 수 있는 함수 입니다.





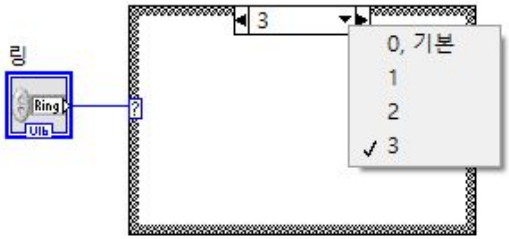
여기서는 선택기를 숫자로 만들었습니다.

그러나 링, 열거형을 이용하면 문자로 이루어진 선택기를 사용할 수 있습니다.

링, 열거형

열거형 사용

- 열거형은 U16 숫자형 터미널로 표현되나 문자열과 숫자 값에 대한 정보도 포함됩니다.



링 사용

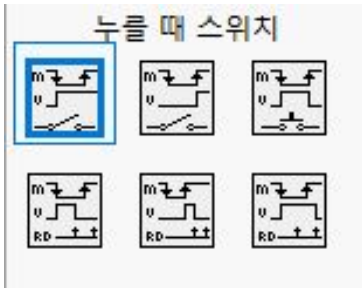
- 반면 링은 문자열 정보 없이 숫자형 터미널로만 표현됩니다.
- 또한 링은 데이터 형을 U16 뿐만 아니라 다른 것으로도 변경할 수 있습니다.
- 링은 텍스트 링, 메뉴 링, 그림 링, 시스템 링이 존재합니다.

열거형, 텍스트 링 사용하기

텍스트 링을 우클릭하고 아이템 편집을 누릅니다.

아이템에 텍스트를 입력하고, 삽입으로 아이템을 추가 할 수 있습니다.

불리언



불리언은 참과 거짓을 입력하는 데이터 타입으로 주로 논리연산에서 사용됩니다.
불리언의 고유 특징으로는 기계적 동작이라는 옵션이 있다는 것입니다.

기계적 동작

스위치 : 눌려진 상태나 올라간 상태를 유지합니다.
래치 : 불리언을 누를 때만 반응합니다.

기계적 동작은 스위치 3개, 래치 3개로 총 6종류가 존재합니다.

또한 정지 버튼과 확인 버튼은 기본적으로 놓을 때 래치로 설정되어 있습니다.

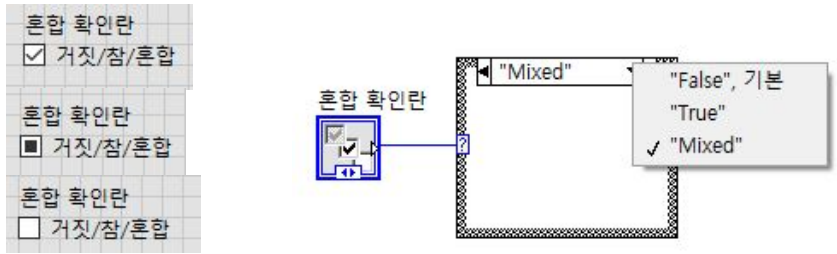
라디오 버튼

라디오 버튼은 한 번에 하나씩 선택할 수 있는 불리언 구조입니다.

라디오 버튼의 특징은 다음과 같습니다.

- 클러스터의 구조를 가지고 있으나 데이터 타입은 열거형으로 취급된다.
- 라디오 버튼 내에 불리언을 마음대로 넣을 수 있다.
- 삽입된 불리언의 순서를 재설정 할 수 있다.

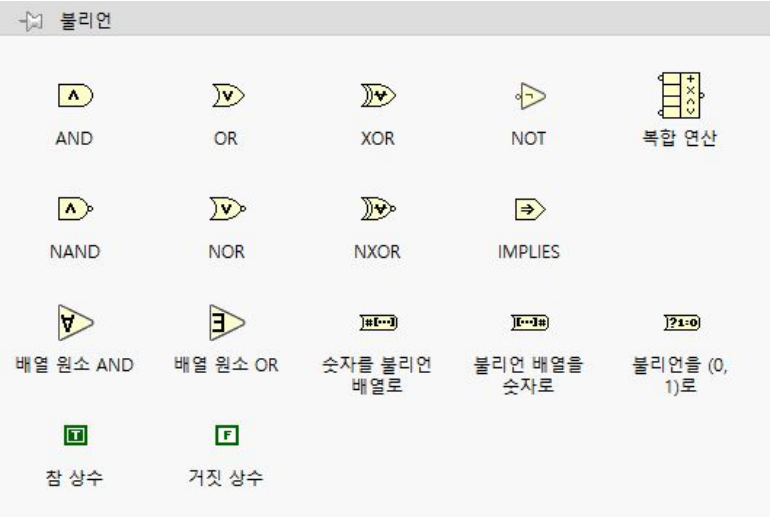
혼합된 확인란



혼합된 확인란은 참/거짓으로 이루어진 불리언과 달리 참/거짓/혼합이라는 3가지 선택지를 가지고 있습니다.

라디오 버튼과 마찬가지로 열거형 데이터이며, 팔레트 – 시스템 – 불리언에서 찾을 수 있습니다.

불리언 함수



불리언 함수는 논리 연산을 위한 다양한 함수들을 지원합니다.

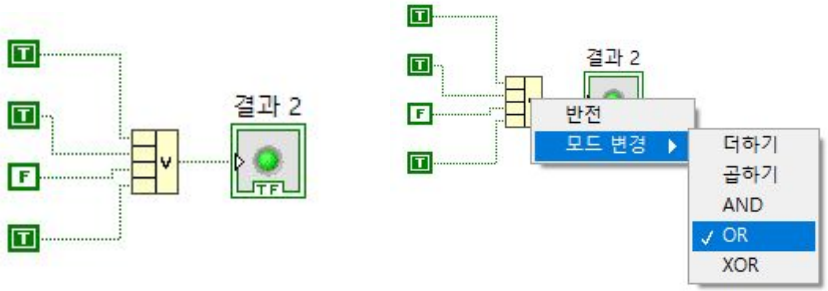
AND, OR, XOR, NOT, NAND, NOR, NXOR, IMPLIES : 논리 연산자입니다.
복합연산 : 숫자형에서 반복 연산에 사용했던 그 복합연산이랑 똑같습니다.
배열 원소 AND, OR : 불리언 배열로 AND, OR 연산을 진행합니다.

숫자를 불리언 배열로, 불리언 배열을 숫자로 : 101010과 같은 숫자로 불리언 배열을 제어합니다.
불리언은 0.1로 : 불리언은 숫자로 바꿉니다.

참, 거짓 상수 : 불리언 상수입니다.

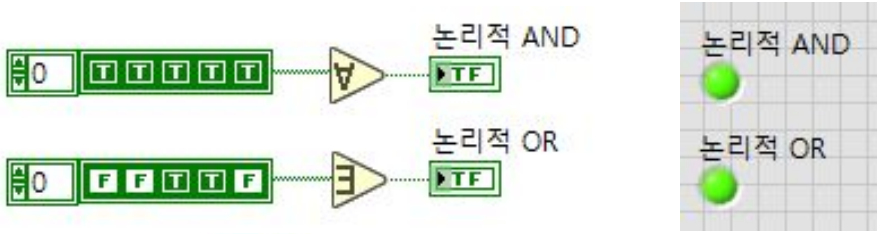
복합연산

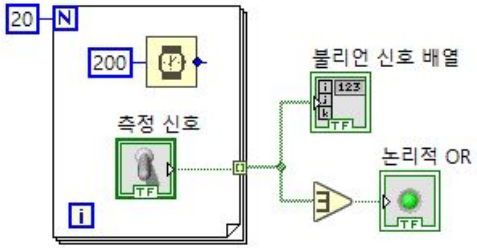
- 숫자형에서 반복 연산에 사용했던 그 복합연산이랑 똑같습니다. 사용법도 동일합니다.
- 옵션 변경으로 숫자형에서 쓰던 복합 연산으로 바꿀 수 있습니다.



배열 원소 AND, OR (예시 참고)

- 불리언 배열로 AND, OR 연산을 진행합니다.
- 논리적 AND, 논리적 OR이라고도 부릅니다.
- 배열에 있는 모든 원소들로 논리 연산을 한번에 진행합니다.
- 즉, 들어온 모든 값들 중 전부 참인지, 하나라도 참이 있는지 검토하는 데 유용합니다.

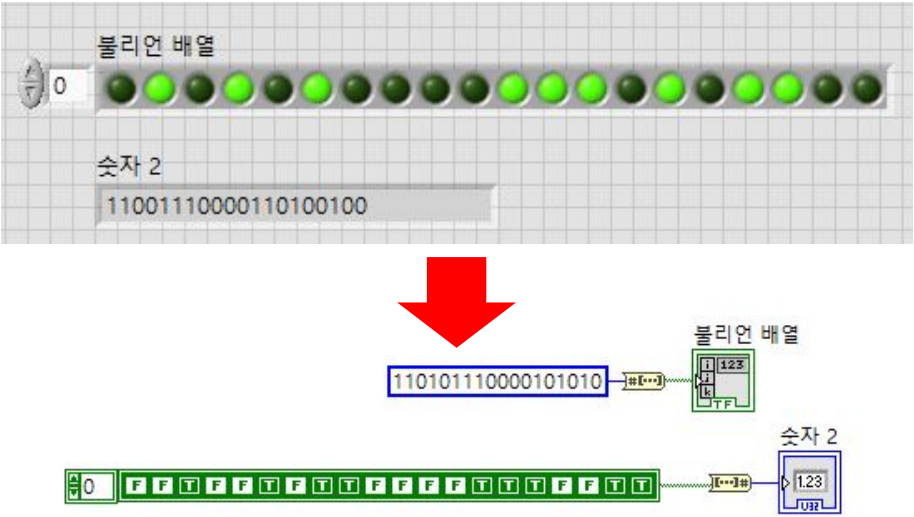




예시

숫자를 불리언 배열로, 불리언 배열을 숫자로

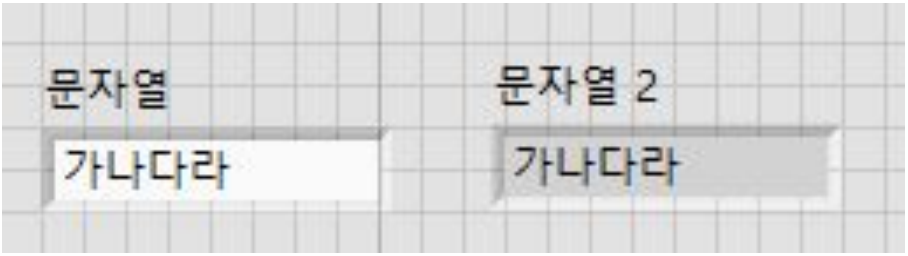
- 2진수를 불리언 배열로, 불리언 배열을 2진수로 변경합니다.
- 10진수로 입력하든 16진수 입력하든 들어가는 값은 2진수로 변환되어서 들어갑니다.
- 때문에 사용 전에 숫자형의 기수를 2진수로 바꿔놓는 것을 권장합니다.
- 서로 뒤바뀌어 있는 순서에 주의 합시다!

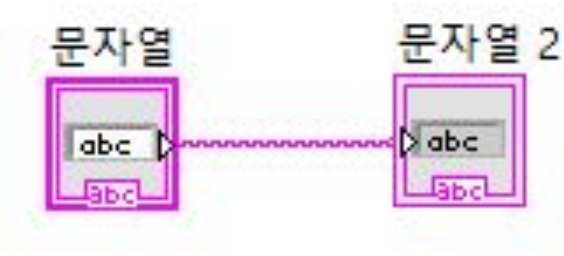


불리언은 0.1로

- 불리언은 숫자로 바꿉니다.
- 참은 1로, 거짓은 0으로 바꿉니다.

문자열





문자열은 컨트롤 – 일반 – 문자열 & 경로 팔레트에서 불러올 수 있습니다.

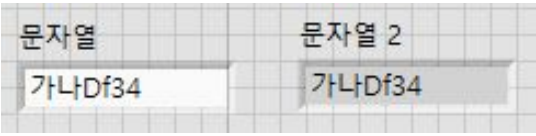
문자열은 아스키코드로 표현됩니다.

후에 설명할 문자열 함수들을 사용하려면 가능하면 문자열은 영어로 작성하는 것을 권장합니다.

문자열

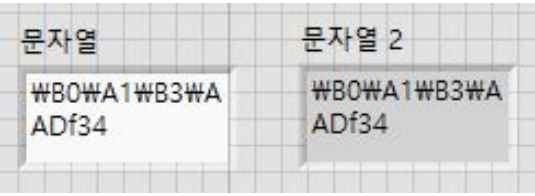
일반 디스플레이

- 문자를 그대로 출력



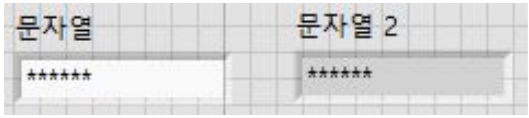
\ 코드 디스플레이

- \코드로 문자를 출력



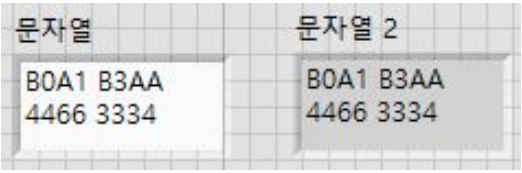
암호 디스플레이

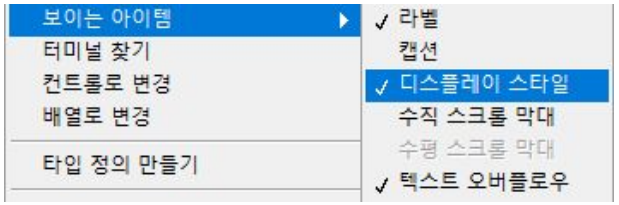
- 문자를 안 보이게 출력



16진수 디스플레이

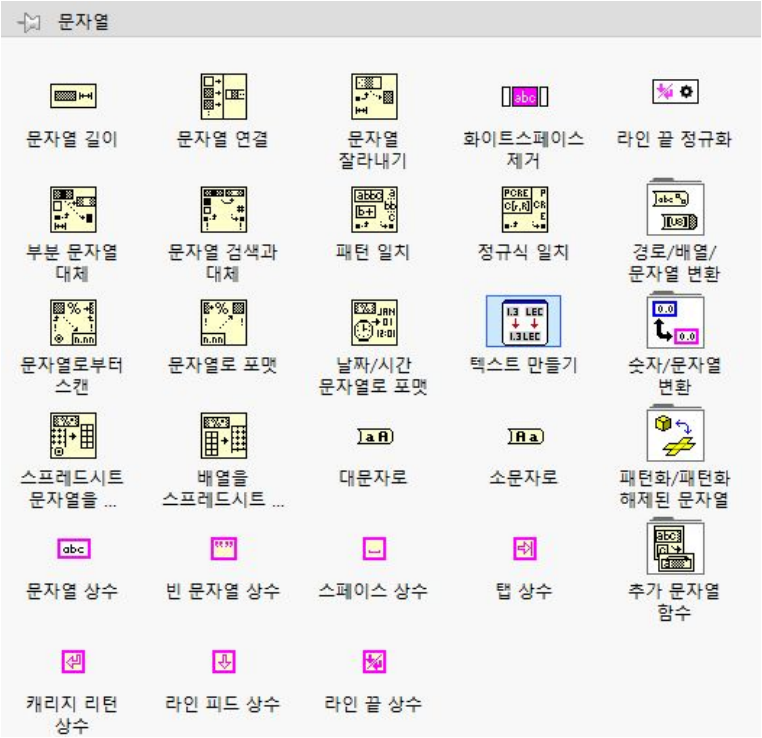
- 문자를 16진수로 출력





우클릭 – 보이는 아이템 – 디스플레이 스타일로 표시 옵션을 확인 할 수도 있습니다.

문자열 함수

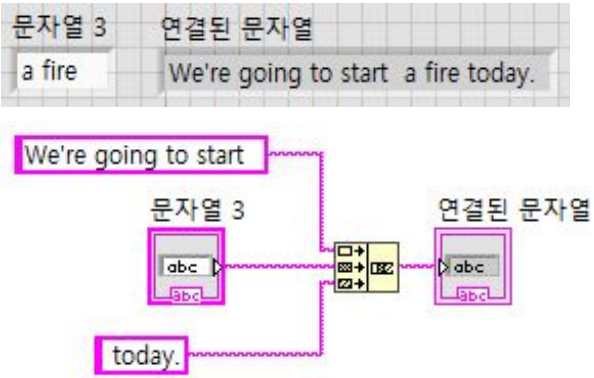


자주 사용하는 것 위주로 배워보도록 합시다.

문자열 함수를 쓰기 위해서 문자열은 기본적으로 영어로 작성하는 것을 권장합니다.

문자열 연결

- 별도의 문자열들을 연결하여 하나의 문자열 결과물로 출력합니다.

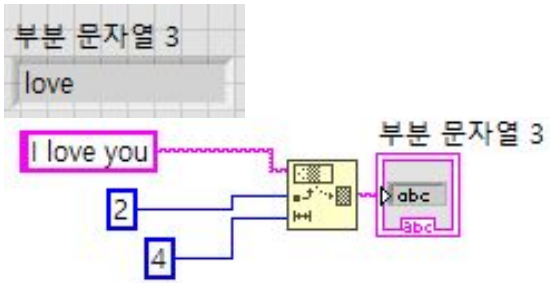


숫자 문자열 변환

- 문자열에 적힌 숫자를 진짜 숫자로 바꿉니다.
- 혹은 숫자를 문자열로 변환해서 출력합니다.

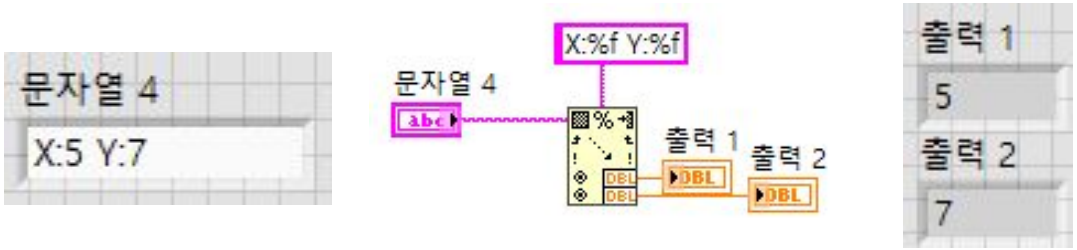
문자열 잘라내기

- 문자열에서 위치와 길이를 지정해, 특정 문구만 추출합니다.



문자열 함수를 쓰기 위해서는 은 기본적으로 영어로 작성하는 것을 추천합니다.

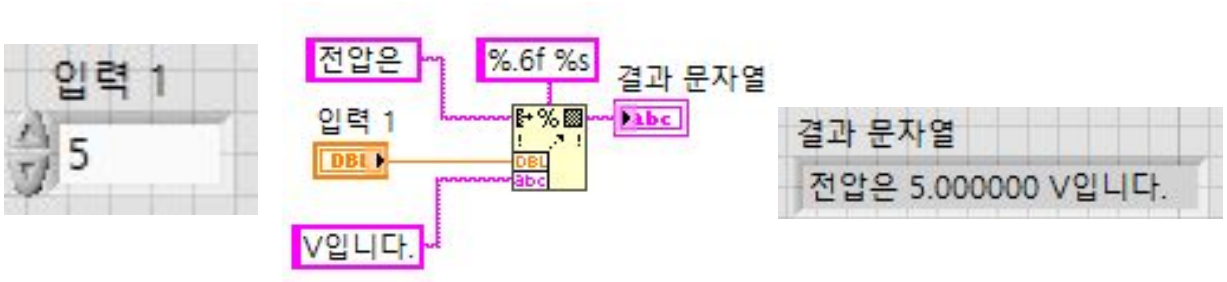
문자열로부터 스캔



문자열을 스캔해서 원하는 값 만을 추출합니다. %f는 실수, %d는 정수입니다.

입력 문자열와 포맷 문자열의 구문은 동일해야만 합니다,

문자열로 포맷



입력 된 데이터들을 조합해서 포맷 문자열로 지정한 포맷의 문자열로 만들어 줍니다

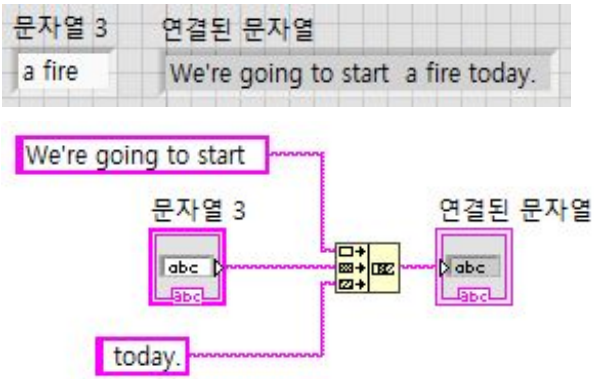
입력 문자열와 포맷 문자열의 구문은 동일해야만 합니다.

실습 : 문자열로 받아 온도 값을 정수로 추출하고, 그 값을 화씨로 바꾸시오.

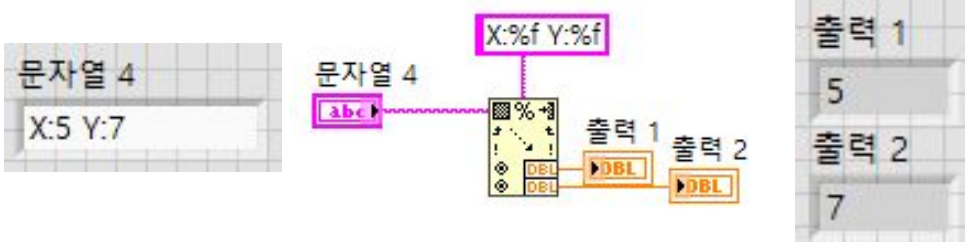
$$°F = °C \times 1.8 + 32$$

- 숫자 65를 입력하십시오.('10진수 문자열을 숫자로 함수' 사용)
- 문자열로 The current temperature is 65.(현재 온도는 65도입니다.)를 받으십시오
- 그리고 65란 문자를 추출하십시오
- 문자열 65를 숫자로 바꾸시오.
- 그리고 65도를 섭씨를 화씨로 바꾸시오,

힌트!

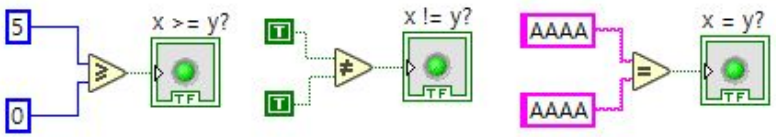


+



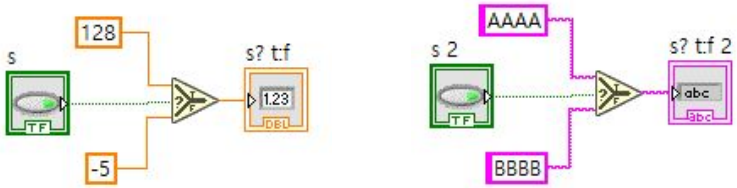
비교

비교 함수



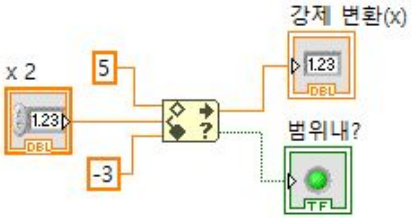
비교 함수는 두 개의 입력 값을 비교하여 불리언 값을 출력합니다. 입력 하는 값의 형식만 맞으면 어떤 값이든 비교할 수 있습니다.

선택



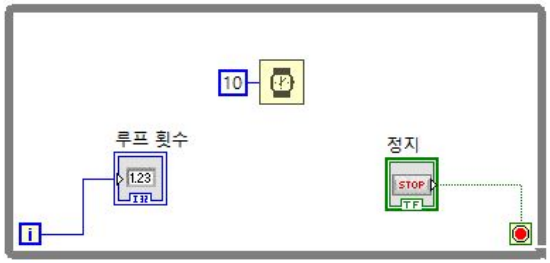
입력 값이 참이냐 거짓이냐 에 따라서 출력되는 값을 선택 할 수 있습니다.

강제 변환



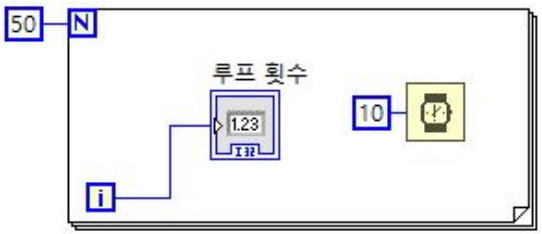
입력값을 일정 범위내에 있도록 강제 변환하거나 범위내에 있을 때 참을 출력합니다.

While 루프



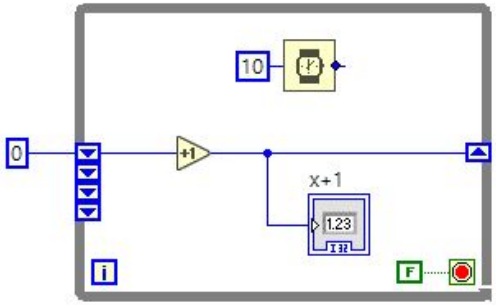
While 루프는 별도의 정지조건이 생길 때까지 무한히 돌아갑니다.
기본적으로 이 안에서 프로그래밍을 진행합니다.
I 아이콘은 루프 횟수, 빨간 아이콘은 정지 조건으로 참이 들어가면 루프가 정지합니다.
루프는 매우 빠르게 돌아가므로 기다림 함수를 넣길 권장합니다.

For 루프



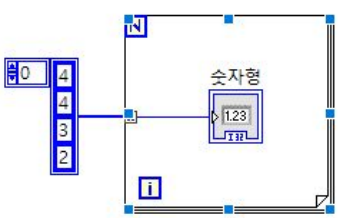
For 루프는 지정한 횟수만큼만 돌아갑니다.
I 아이콘은 루프 횟수, N 아이콘은 정지 조건으로 정해진 숫자만큼 돌아가면 루프가 정지합니다.
조건 터미널 옵션으로 중간 정지가 가능합니다.

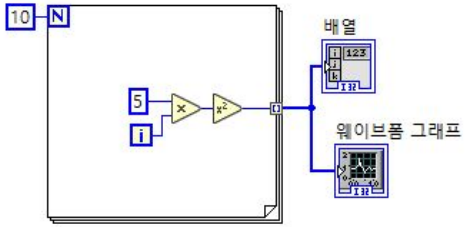
시프트 레지스터



While루프와 For 루프에서 추가되는 요소로, 루프가 끝날 때, 이전의 값을 저장해서 루프가 다시 시작할 때 출력시킵니다.

루프의 입출력

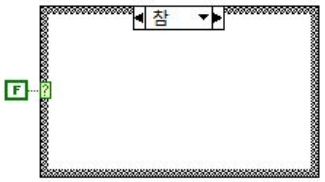


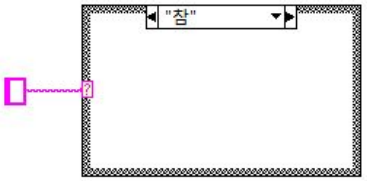


인덱싱

- For 루프에 배열을 입력하고 인덱싱 기능을 사용하면 순차적으로 상수를 부릅니다.
- For 루프에서 상수를 출력하고 인덱싱 기능을 사용하면 순차적인 배열을 만듭니다.

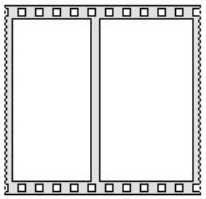
케이스 구조

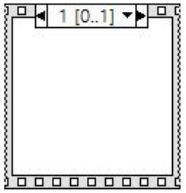




특정 조건에 맞춰 함수나 숫자를 실행 시킬 수 있습니다. 케이스는 다양한 조건을 사용할 수 있습니다.

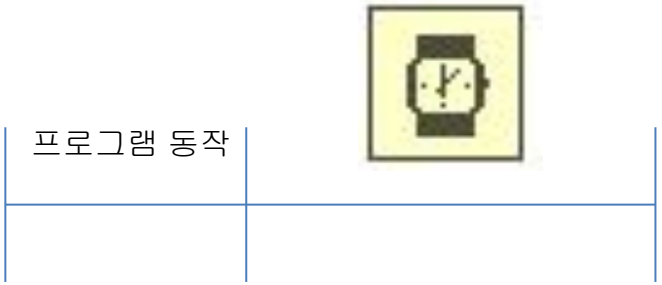
시퀀스 구조





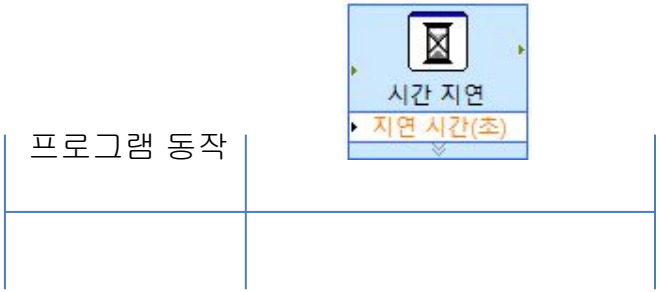
시퀀스 구조는 순차적인 프로그램 구성을 만들기 위해 사용됩니다.
시퀀스 구조는 다층 시퀀스로 압축시킬 수 있습니다.

기다림



기다림 함수는 이름 그대로 잠시 동작 시간을 지연시키는 용도로 사용하는 함수입니다.
ms 단위로 사용합니다.

시간지연

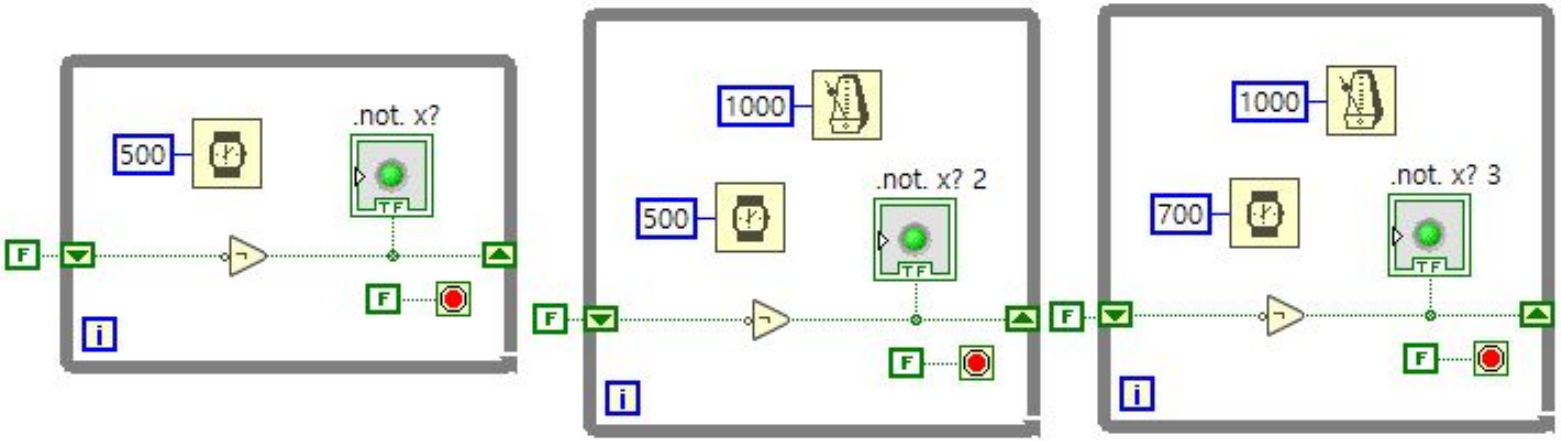


시간 지연은 기다림 함수와 기능은 같으나 초 단위 사용합니다.

다음 ms배수까지 기다림

While 루프에서만 사용할 수 있는 함수이며, 루프의 전체 시간을 결정하는 함수입니다.

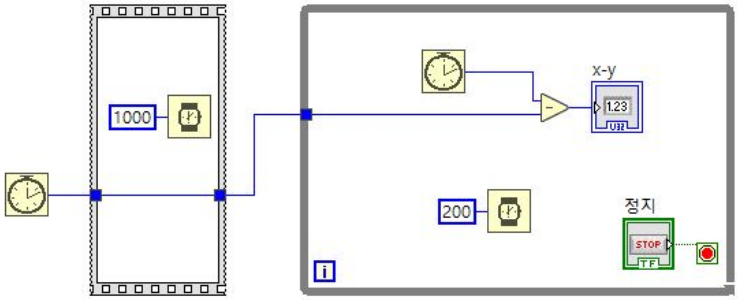
실습. 기다림과 다음 ms배수까지 기다림의 차이를 알아보자



- 기다림 500ms와 NOT+시프트 레지스터로 점멸하는 LED를 만듭니다.
- 기다림 500ms와 다음 ms배수 1000, NOT+시프트 레지스터로 점멸하는 LED를 만듭니다.
- 기다림 700ms와 다음 ms배수 1000, NOT+시프트 레지스터로 점멸하는 LED를 만듭니다.

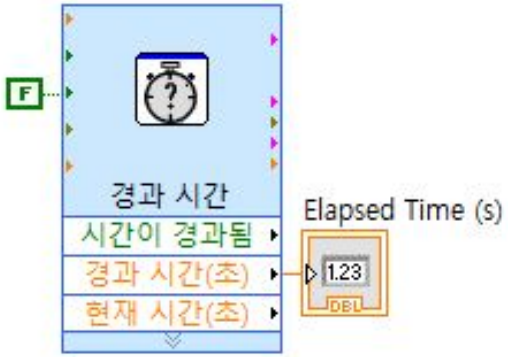
- 다음 ms배수에 의해 전체 루프 시간이 결정 되었으므로 기다림 함수가 추가되어도 루프는 반드시 1000ms에 종료됩니다.

틱 카운트 , 높은 분해능



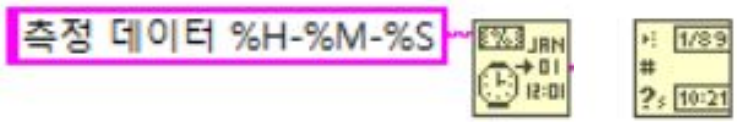
- 틱 카운트와 높은 분해능은 2개의 함수를 빼는 것으로 프로그램의 동작시간을 알 수 있습니다.
- 이를 응용하여 프로그램 코드의 연산 시간을 구체적으로 파악할 수 있습니다.
- 틱 카운트는 기다림 함수의 영향을 받지 않고 독자적인 카운트를 수행합니다.
- 틱 카운트와 높은 분해능은 기능과 사용법이 똑같습니다.
- 다만, 틱 카운트는 ms, 높은 분해능은 micro s까지 특정이 가능합니다.

경과 시간



- 경과 시간 함수는 경과한 시간을 측정합니다.
- 일정시간이 지나면 시간을 리셋 시키도록 설정할 수 있습니다.
- 경과 시간 함수를 위 그림처럼 설정하면 틱 카운트와 유사하게 사용할 수 있습니다.

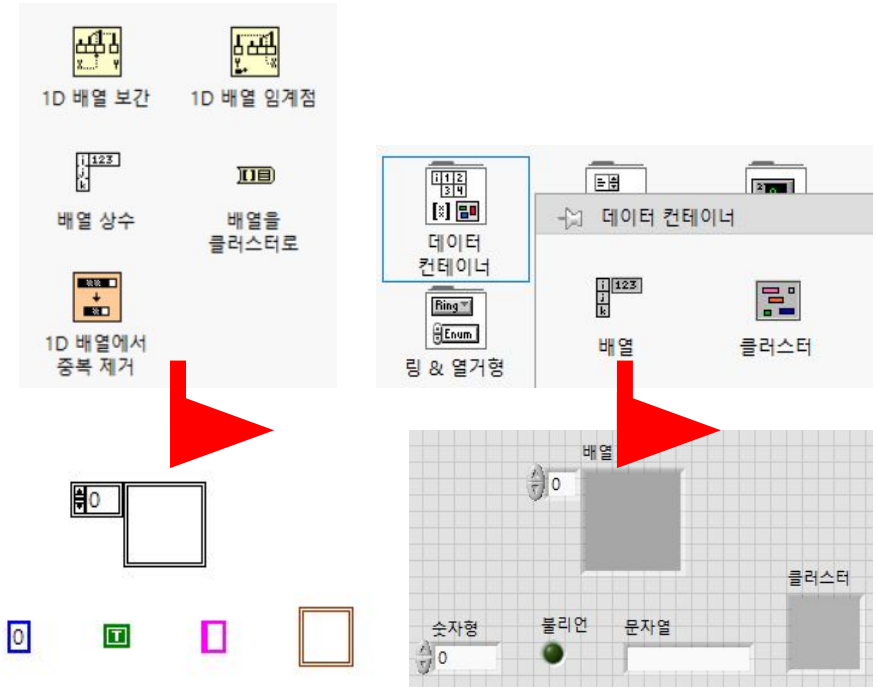
날짜 시간 문자열로 얻기



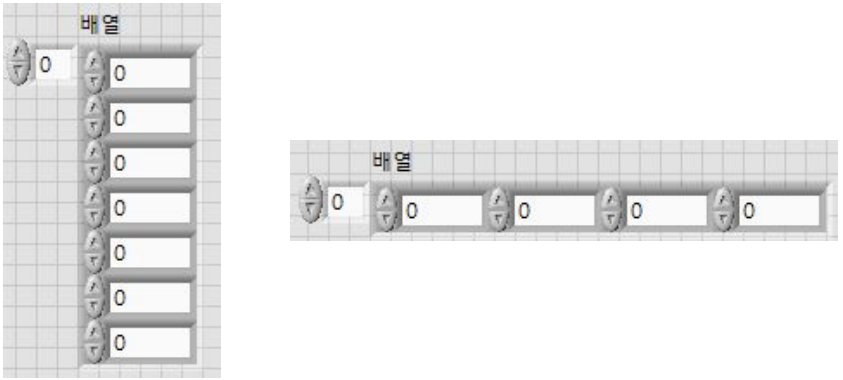
날짜 시간 정보를 문자열로 얻을 수 있다.

함수에 따라서는 출력 포맷을 수정할 수 있다.

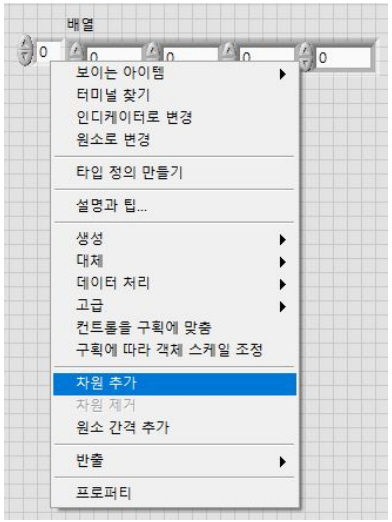
https://zone.ni.com/reference/ko-XX/help/371361R-0129/glang/codes_for_time_format_str/

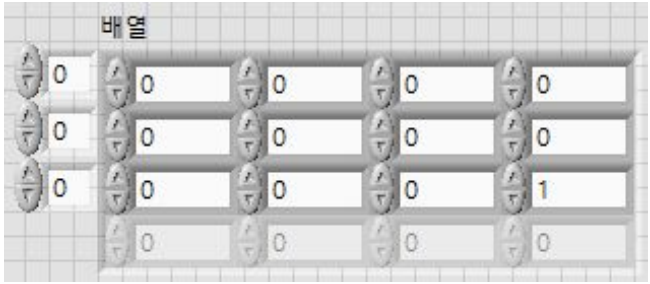


프런트 패널 혹은 블록다이어그램에서 배열을 불러오고, 배열 박스 안에 숫자형, 불리언, 문자열, 클러스터 등을 삽입할 수 있다.

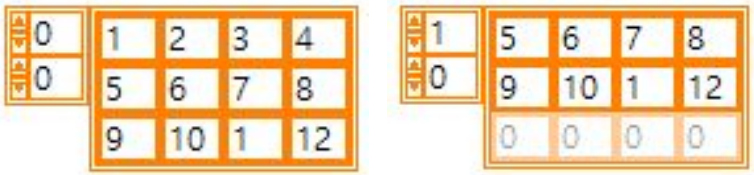


배열의 순서는 위에서부터, 그리고 왼쪽부터 시작하며 0번 부터 시작한다.





우클릭으로 배열의 차원을 추가 할 수 있지만, LabVIEW 에서는 지원하는 함수가 2차원까지이므로 대체로 2차원 배열까지만 사용한다.

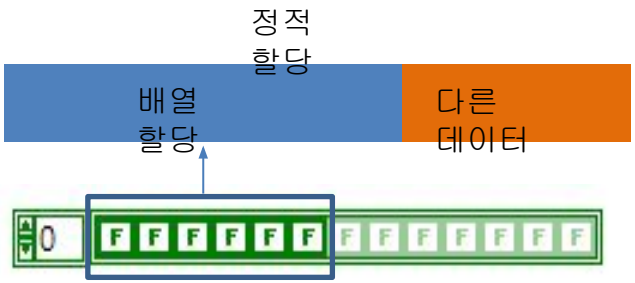


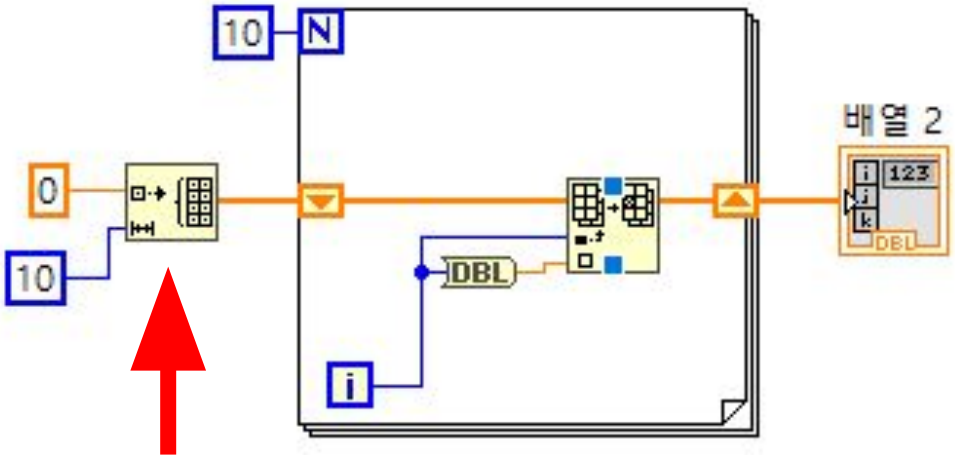
배열 인덱스

- 배열에는 차원당 한 개의 인덱스가 필요합니다.
- 인덱스는 배열을 탐색하는데 사용합니다.

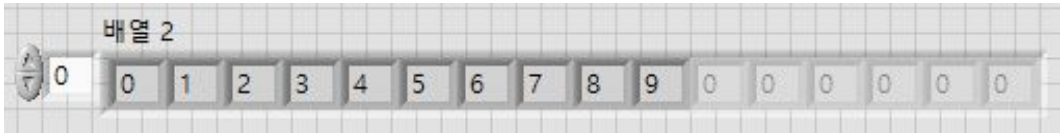
LabVIEW에서 배열은 C언어와 다르게 동적 할당되므로 배열 개수를 정하지 않아도 프로그래밍이 가능하다.

때문에 CPU가 계속 새로운 리소스를 할당해야 하므로 처음에 필요한 만큼의 배열을 미리 선언해두고 프로그래밍 하는 것이 권장된다.





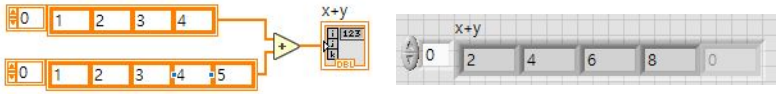
이렇게 미리 배열을 선언하고
사용하는게 바람직하다



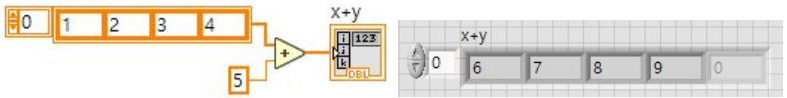
다양한 배열 함수들을 접하다 보면 동적 할당 특성상 배열이 늘어날 수도 있다.
따라서 사전에 배열의 크기를 예측해서 미리 선언을 한 뒤 사용하는 것이
바람직하다.

다행인 것은 단순 배열 연산으로는 배열의 크기가 변하지 않는다는 것이다.

배열 연산

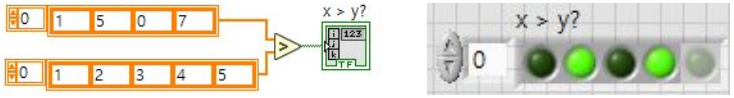


1D 배열 + 1D 배열

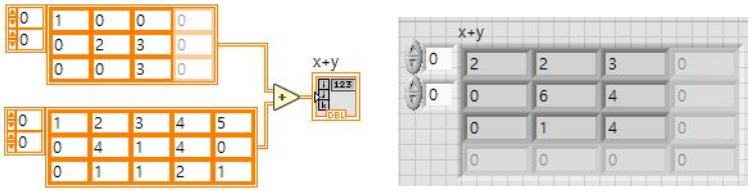


1D 배열 + 스칼라

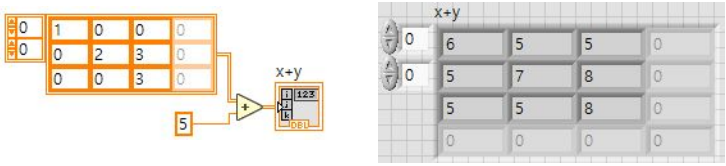
2D 배열 + 1D 배열(불가능)



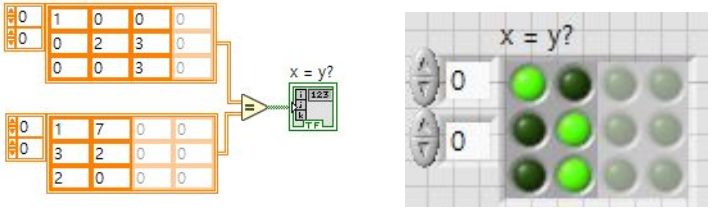
비교



2D 배열 + 2D 배열



2D 배열 + 스칼라

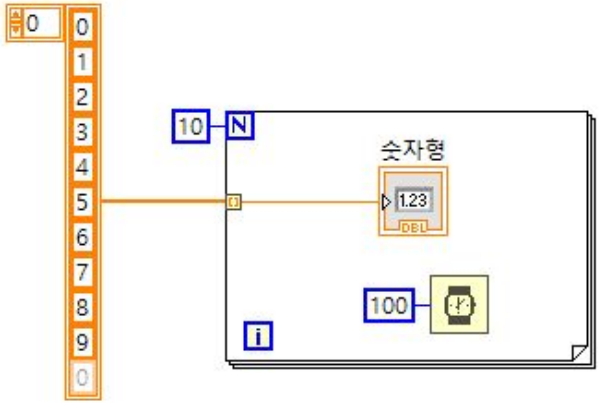


비교

(!) 다행히도 단순 연산만으로는 배열의 크기가 커지지 않는다!

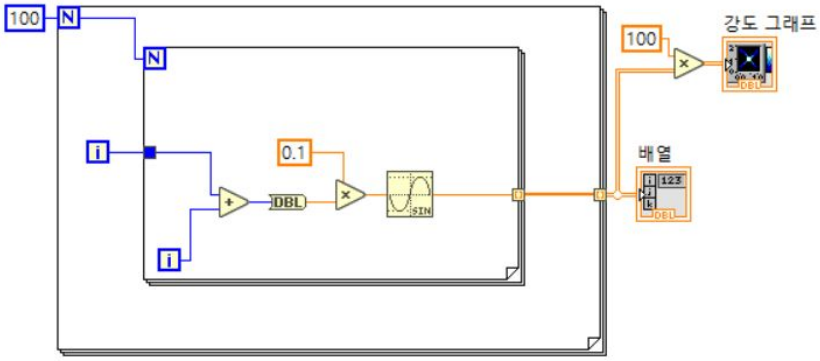
For 루프로 배열 만들기

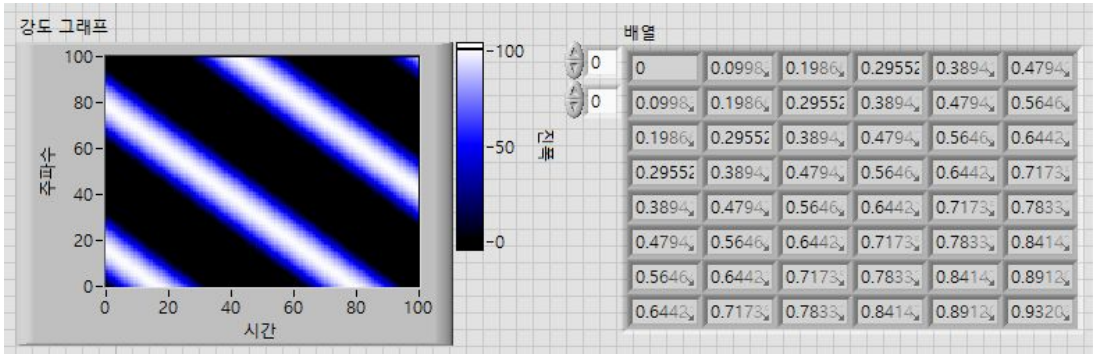
실습 1. 배열을 인덱싱하여 결과값을 관찰하자. + 인덱싱을 없애고 관찰해보자



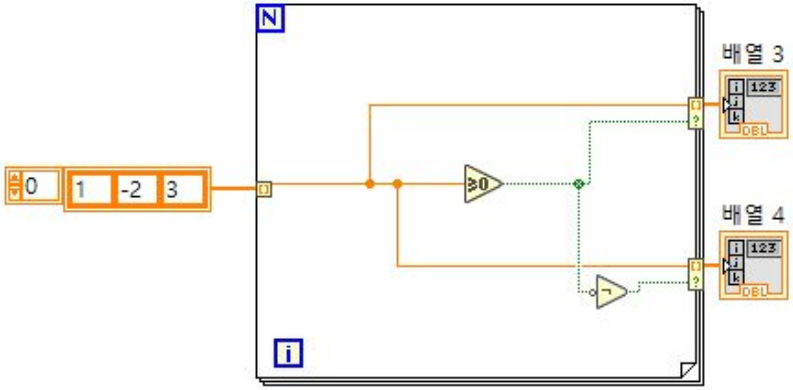
실습 2. 상수를 인덱싱하여 결과값을 관찰하자. + 인덱싱을 없애고 관찰해보자

실습 3. 2중 for문으로 2차원 배열 만들기

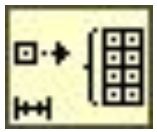




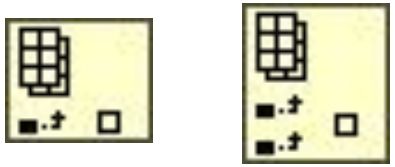
실습 4. for문의 조건적 터널로 숫자 구분하기



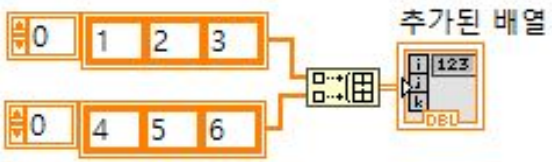
배열 크기 : 배열을 크기를 반환합니다.



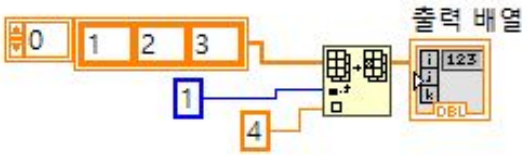
배열 초기화 : 배열을 새로이 생성합니다. 만들어지는 배열은 입력 된 값으로 생성됩니다.



배열 인덱스 : 지정한 부분의 인덱스를 반환합니다.



배열 만들기 : 배열을 조합합니다.(2D, 원소+2D, 1D+2D, 1D+1D 등)

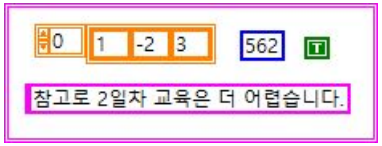


배열 부분 대체 : 인덱스로 배열의 원소나 부분 배열을 대체합니다.

클러스터

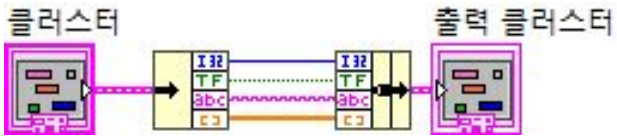
배열과의 차이점

한 종류 원소만 담을 수 있다



숫자, 불리언, 문자열, 배열, 심지어 클러스터도 담을 수 있다.

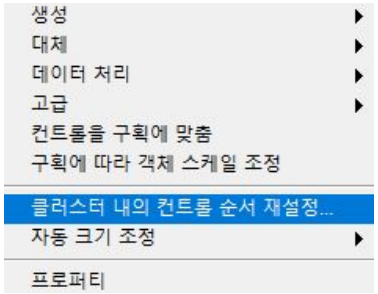
클러스터 묶기 풀기



풀기 : 클러스터를 구성 되어있는 각각의 원소로 분리합니다.

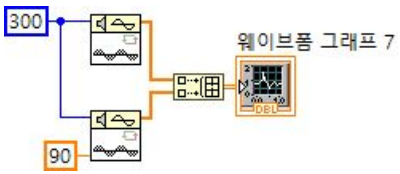
묶기 : 원소들을 집합시켜 새로운 클러스터를 만듭니다.

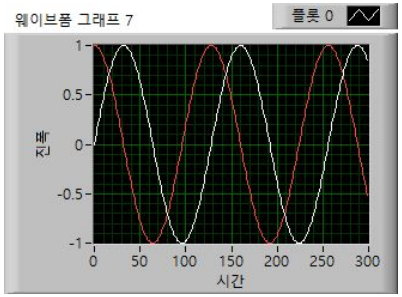
클러스터 내 순서 재설정



클러스터는 순서가 다르면 완전히 다른 클러스터로 취급하므로 주의!

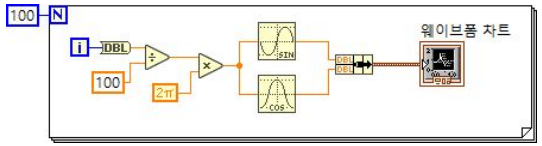
차트와 그래프

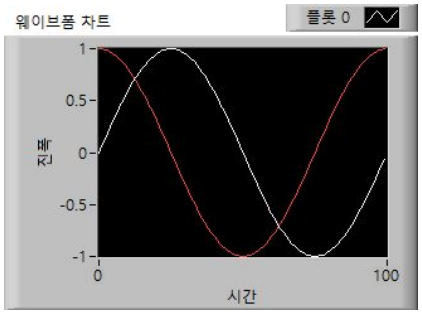




웨이브폼 그래프

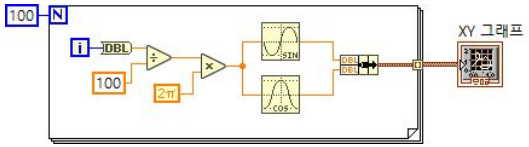
- 배열 데이터를 출력하는 것이 목적입니다.
- 입력 값은 배열입니다.

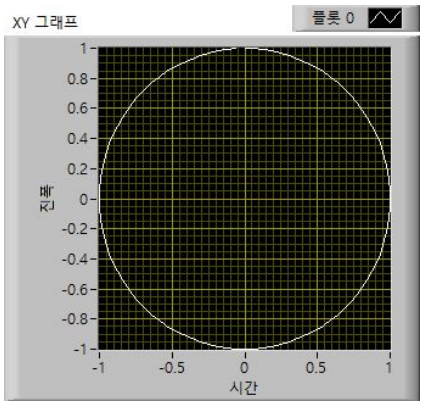




웨이브폼 차트

- 원소의 히스토리를 기록하는 것이 목적입니다.
- 입력 값은 원소입니다.

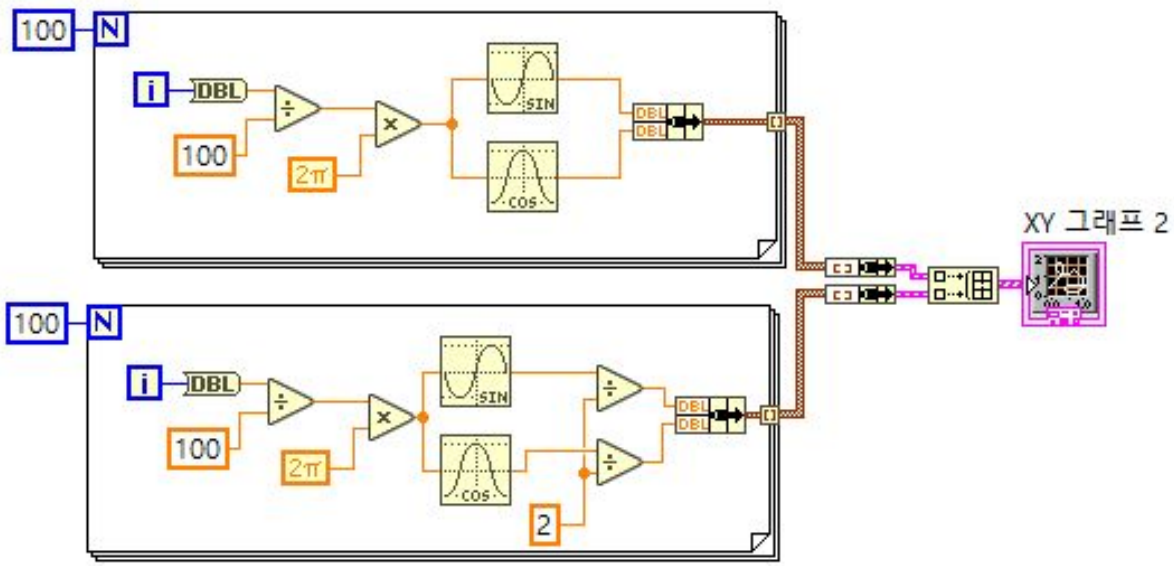


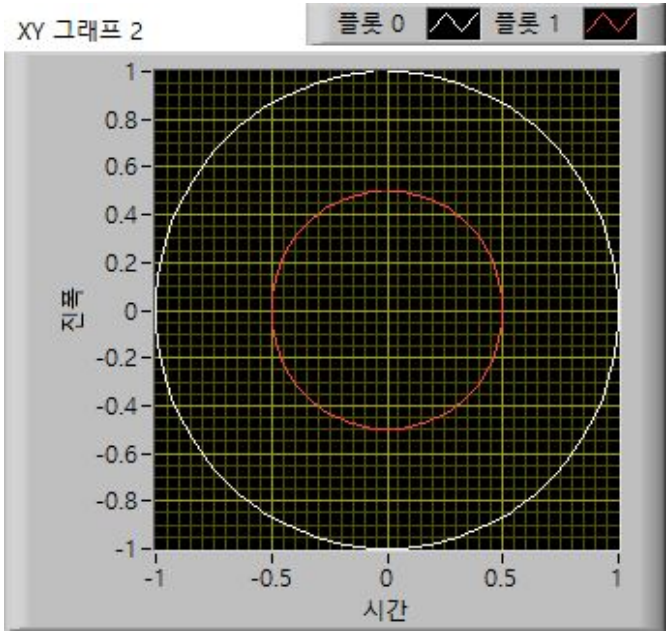


XY 그래프

- X, Y의 값을 출력하는 것이 목적입니다.
- 입력 값은 배열+배열 클러스터입니다.

참고로 X, Y 그래프는 클러스터 + 배열의 조합으로 2개 이상의 플롯을 작성할 수 있습니다.



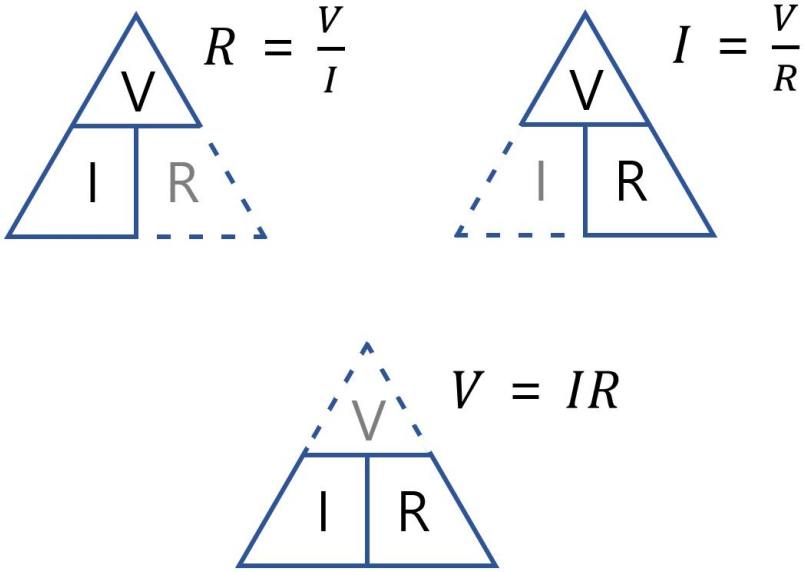


옴의 법칙

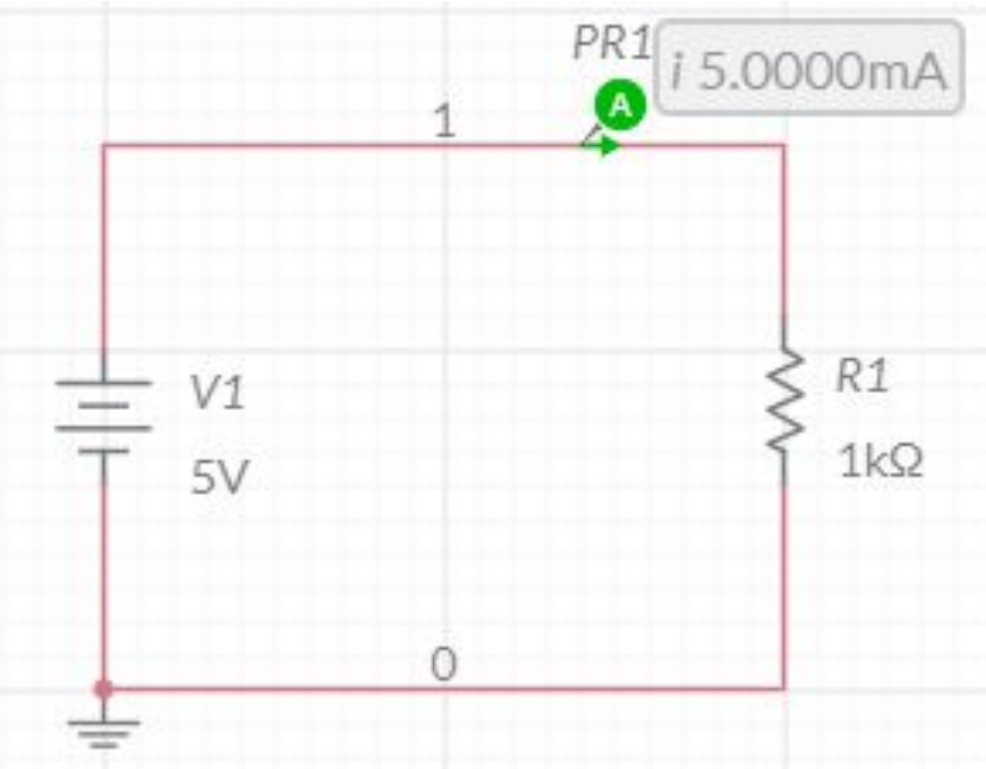
옴의 법칙은 전류와 전압 및 저항 사이의 기본적 관계를 정의합니다.

도체에 일정한 전위차가 존재할 때, 도체의 저항(Resistance)의 크기와 전류(Ampere)의 크기는 반비례하는 법칙입니다.

$$V = IR \qquad I = \frac{V}{R} \qquad R = \frac{V}{I}$$



전류를 측정합시다.



결과 : 5(V) = 0.005(A)*1000(Ω)

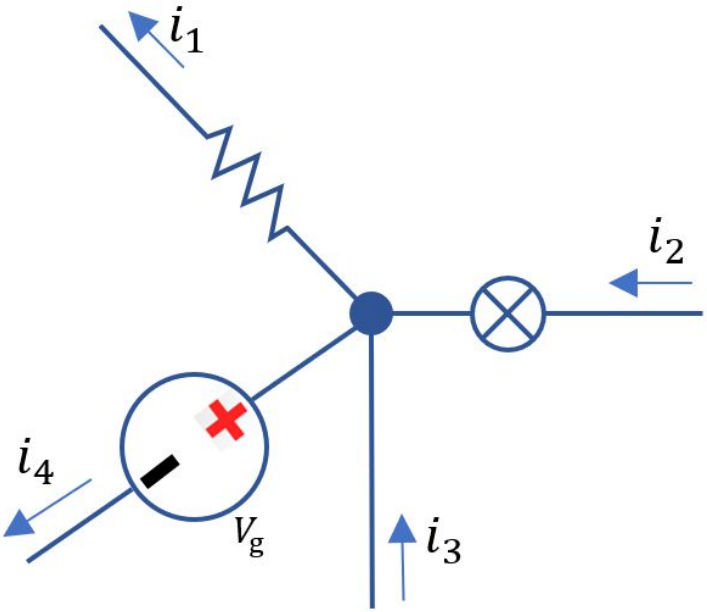
키르히호프 제1법칙

키르히호프 제 1법칙은 키르히호프의 전류 법칙(KCL)이라고 합니다.

KCL은 전하 보존의 원리에 기반합니다.
KCL은 노드로 들어가는 전류의 합이 노드를 나가는 전류의 합과 같다는 것을 암시합니다.

$$(\mathrm{i}_2 + \mathrm{i}_3) - (\mathrm{i}_1 + \mathrm{i}_4) = 0$$

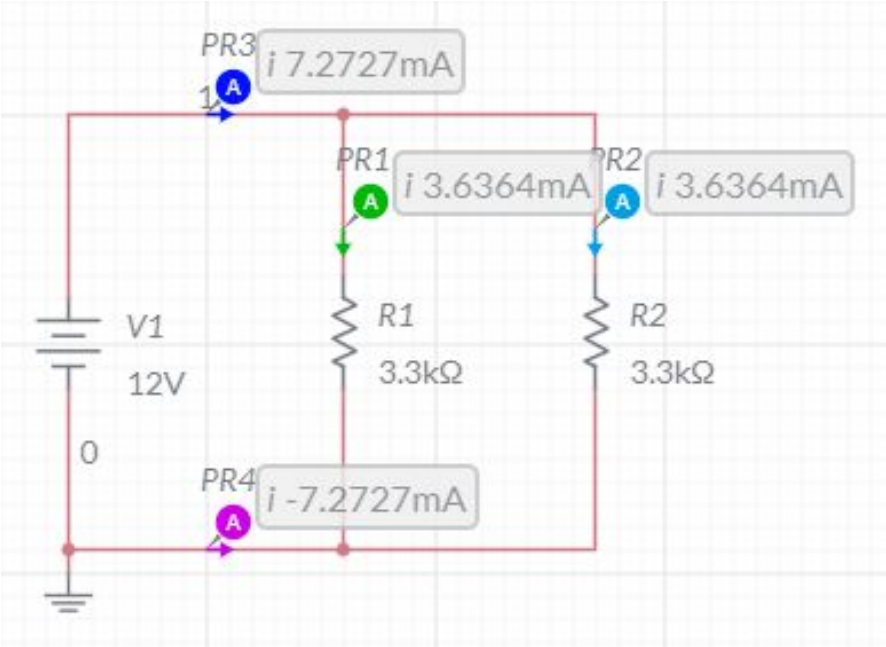
$$\Sigma I_{entering} = \Sigma I_{leaving}$$



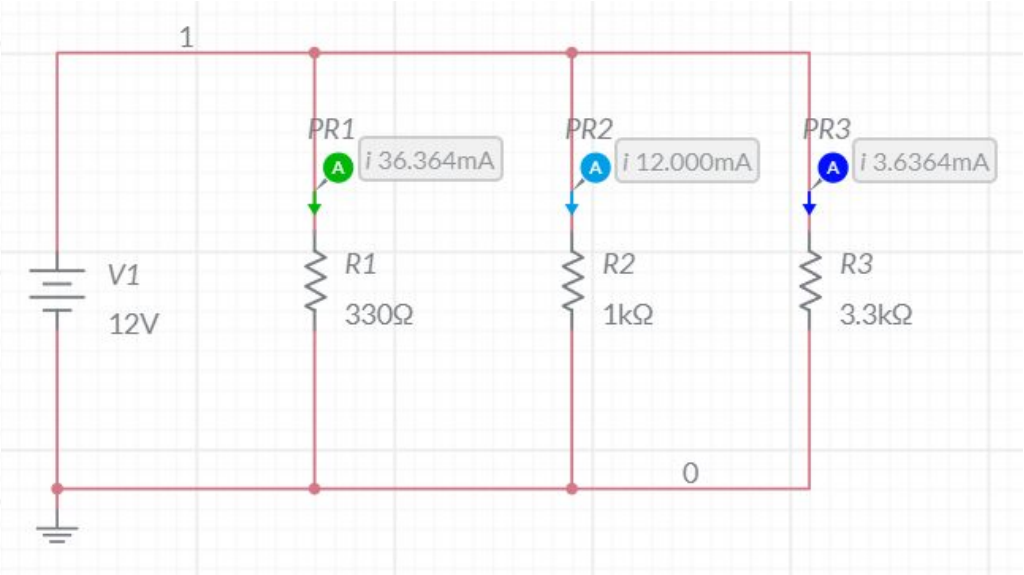
들어오고 나가는 전류의 합은 항상 '0'이 됩니다.

키르히호프 제1법칙

다음의 경우 전류의 흐름이 어떻게 진행되는지 시뮬레이션 해보도록 하겠습니다.



회로 1

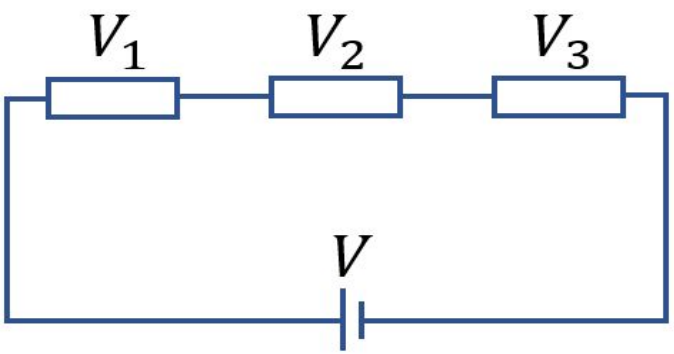


회로 2

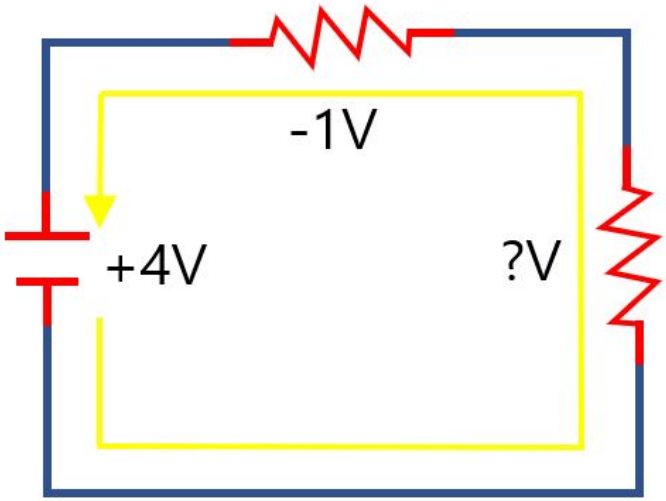
키르히호프
제2법칙

키르히호프 제 2법칙은 키르히호프의 전압 법칙(KVL)이라고 합니다.

KVL은 에너지 보존의 원칙에 기반을 둡니다.
KVL에 따르면 폐쇄 루프 내 전압의 총합은 0입니다.



$$V = V_1 + V_2 + V_3$$

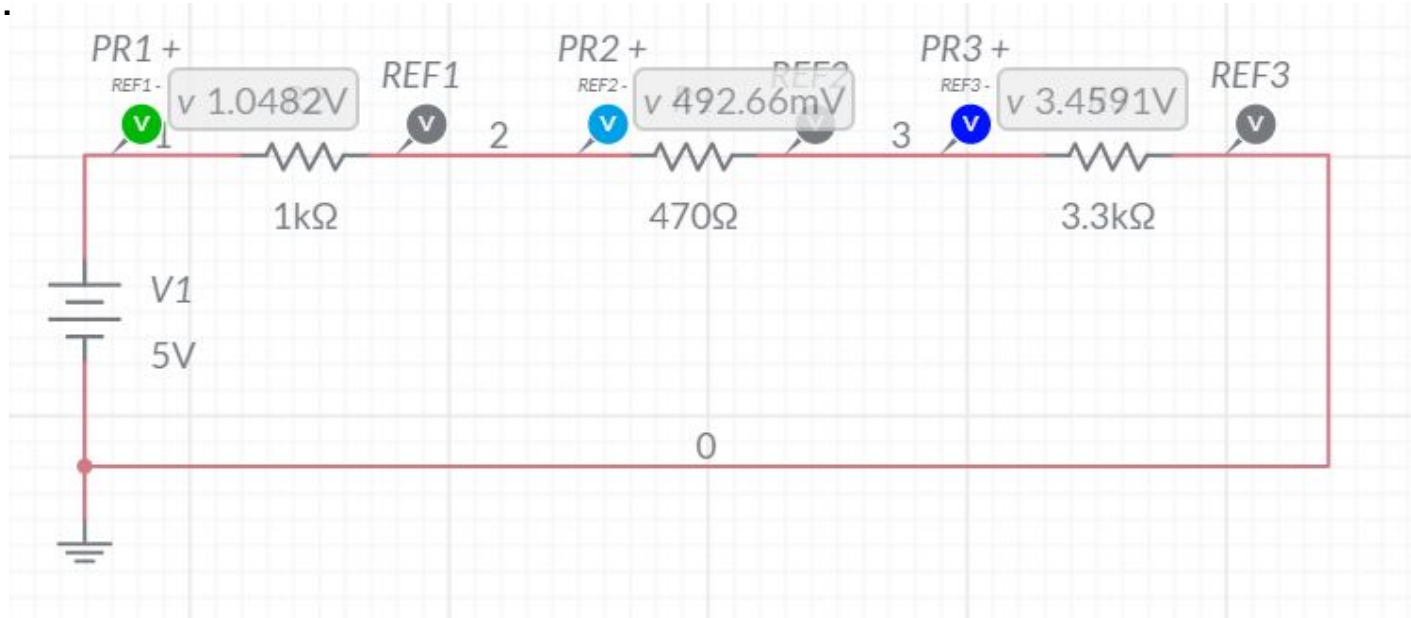


?V(전압강하) = -3V

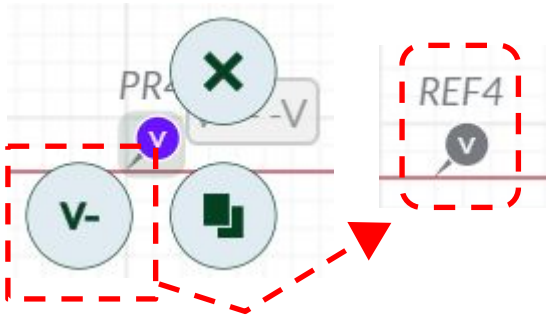
이때 공급된 전압은 양이고 전압 강하는 음입니다.

키르히호프 제2법칙

다음의 경우 전압 분배가 어떻게 진행되는지 시뮬레이션 해보도록 하겠습니다.



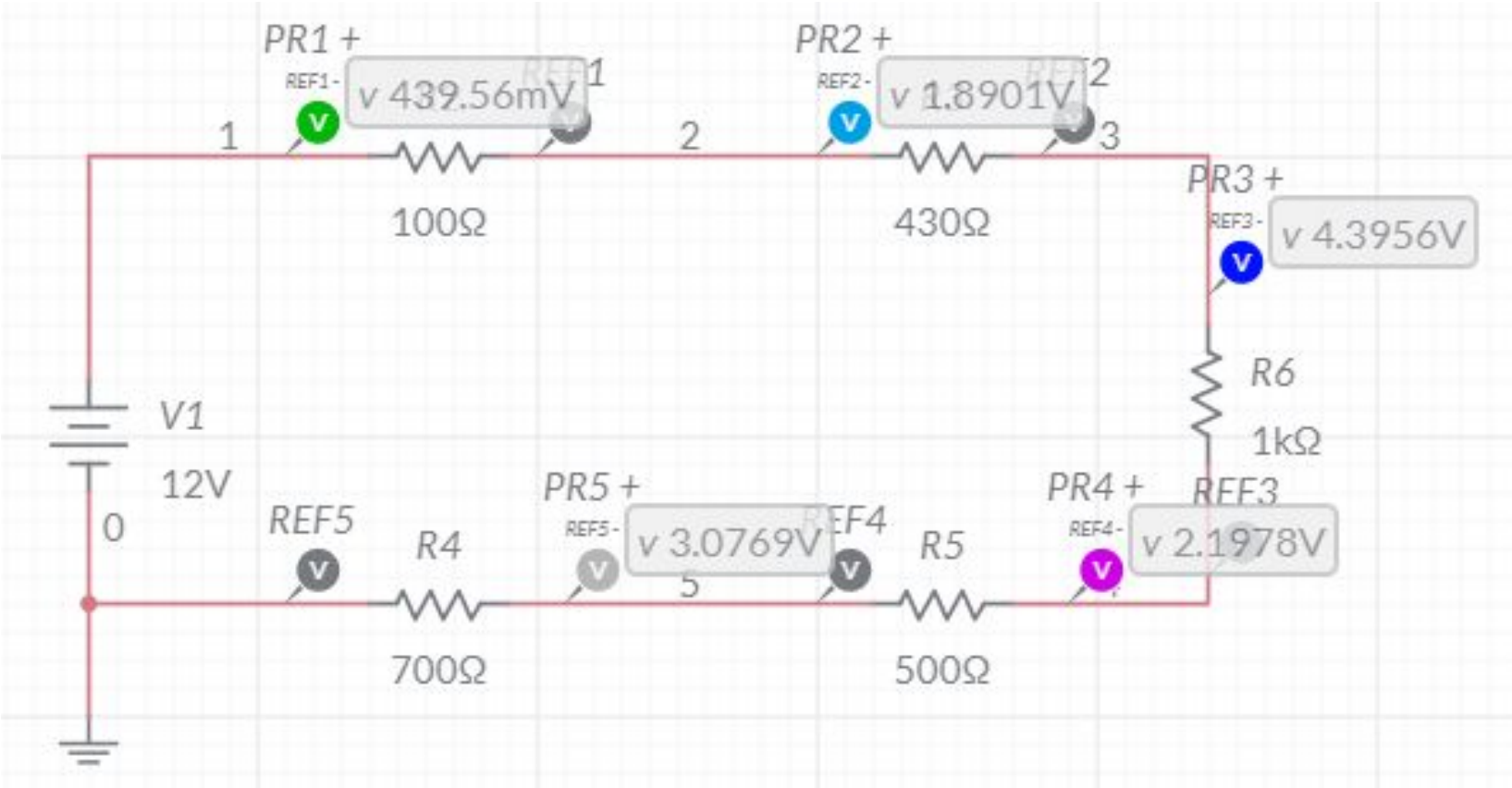
1.0482 + 0.49266 + 3.4591 = 4.99996



정확한 전압 강하 측정을 위해서는 V-를 생성해여 연결해주어야 합니다.

실습(추가과제). 키르히호프 제2법칙

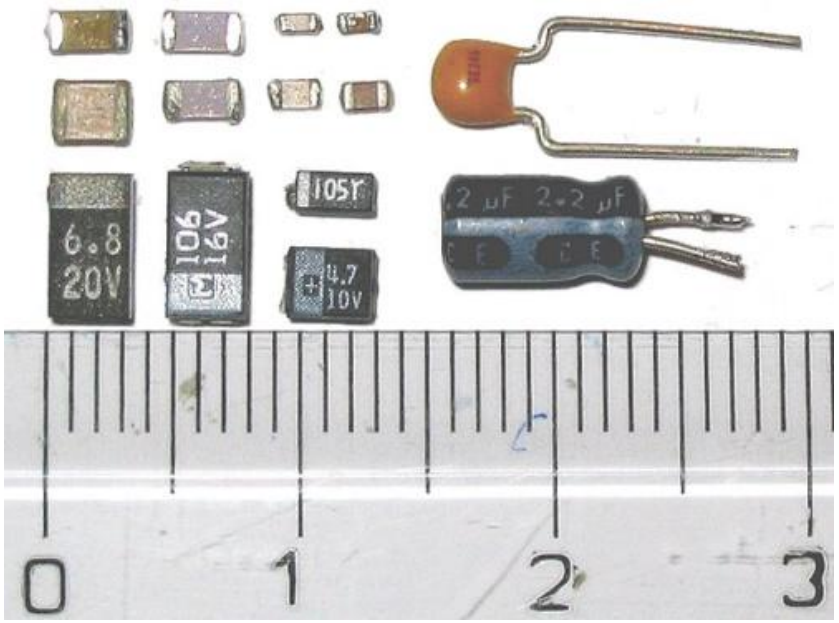
다음의 경우 전압 분배가 어떻게 진행되는지 시뮬레이션 해보고 실제로 측정해보도록 하겠습니다.



커패시터

커패시터는 콘덴서(혹은 축전기)라고 알려져 있으며 전하를 저장하는 장치입니다.
커패시터가 저장할 수 있는 충전량을 커패시턴스라고 합니다.

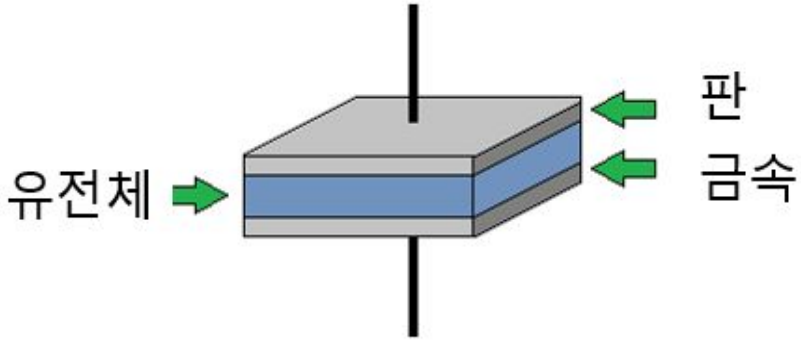
커패시턴스의 단위는 패럿(F)단위이며 1F은 1V전위차에 1C의 전하를 저장하는 것을 말합니다.



커패시터

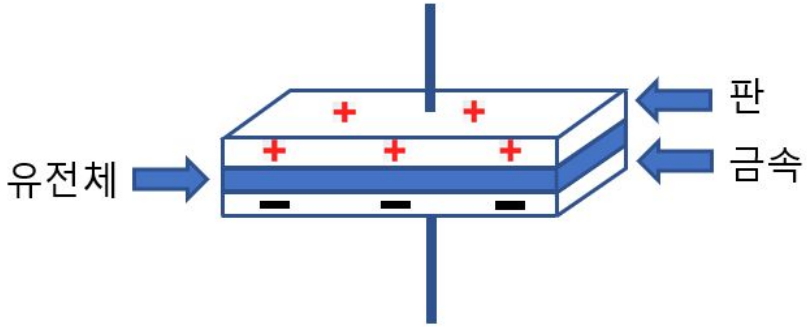
커패시터는 두 개의 병렬 금속판을 가지고 있으며 서로 매우 가까이 배치되어 있고

유전층으로 분리되어 있습니다.



(+)극에 연결된 플레이트는 양전하(Q+)가 모이고 (-)극에 연결된 플레이트는 음전하(Q-)가 모이게 됩니다.

중간의 유전체는 금속 판에 더 많은 전하가 충전 될 수 있도록 도와줍니다.



커패시터

커패시터의 연결 방법은 다음과 같다.

직렬로 연결하면 회로의 총 커패시턴스가 줄어들지만, 내압이 커진다.(견딜 수 있는 전압크기 증가)

직렬 커패시터의 일반적인 용도는 배전시스템, 안전회로

$$C_{EQ} = \frac{1}{\frac{1}{C_1} + \frac{1}{C_2} + \cdots + \frac{1}{C_n}}$$

직렬 커패시터의 총 커패시턴스

커패시터의 전극판의 거리가 늘어난다
= 용량감소 + 견딜 수 있는 전압크기 증가

커패시터

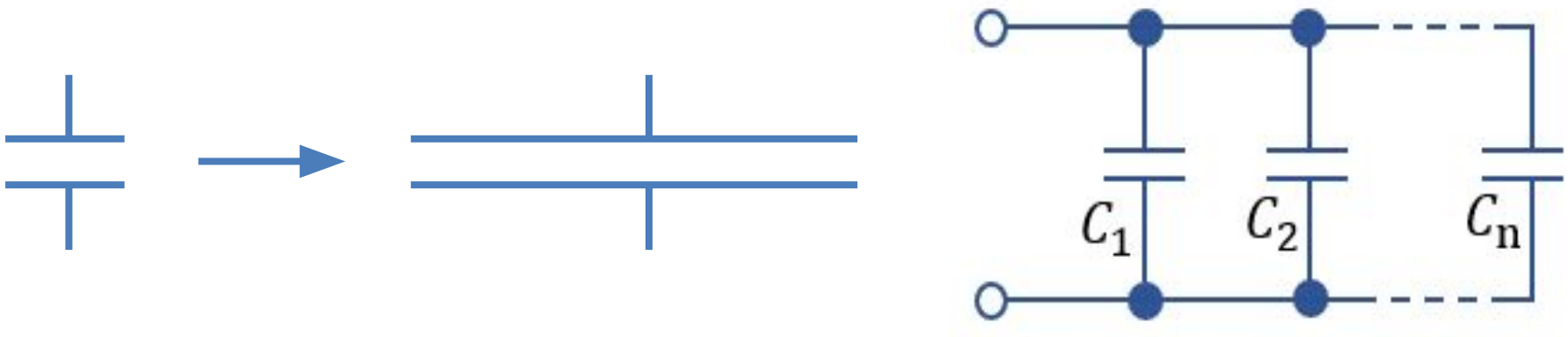
커패시터의 연결 방법은 다음과 같다.

병렬로 연결하면 회로의 총 커패시턴스가 증가한다. 즉, 저장 용량이 증가한다.

병렬 커패시터의 용도는 에너지 저장장치 (예: 고출력 오디오 시스템)와 DC전원의 안정적인 공급

$$C_{EQ} = C_1 + C_2 + \cdots + C_n$$

병렬 커패시터의 총 커패시턴스



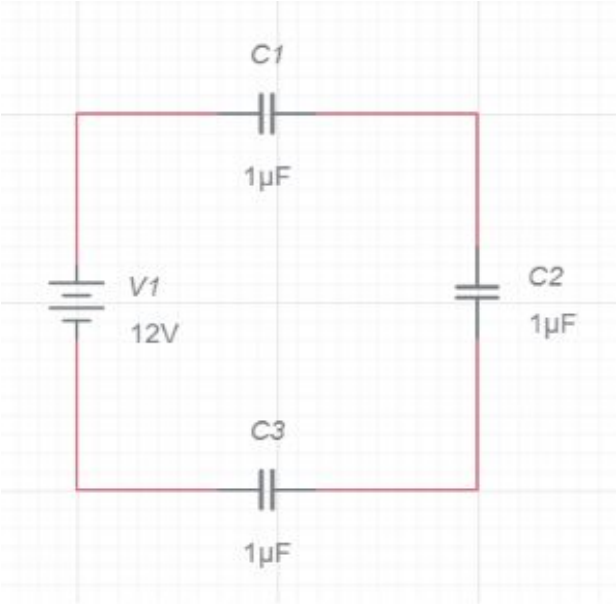
커패시터의 전극판의 면적이 늘어난다.
= 용량증가

실습1. 직렬 커패시터

다음의 직렬 커패시터 회로를 만들고 커패시턴스를 측정해봅시다.

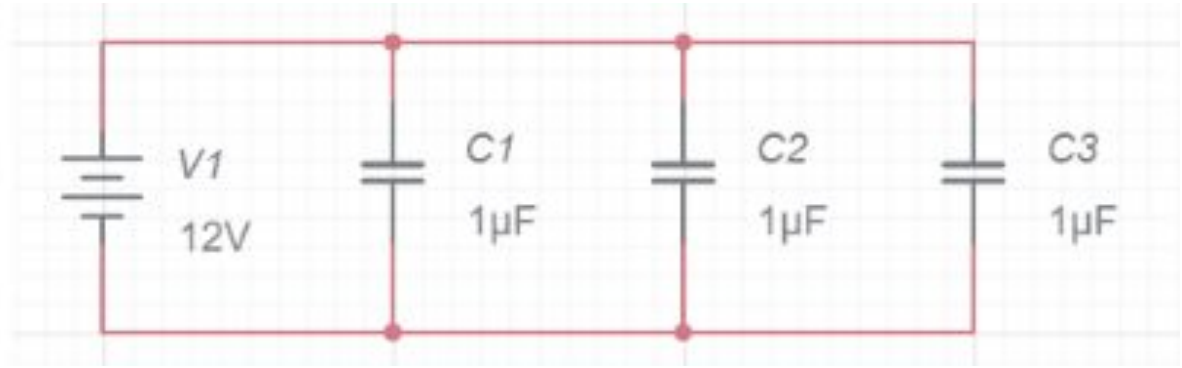
직렬 커패시터의 총 커패시턴스

$$C_{EQ} = \frac{1}{\frac{1}{C_1} + \frac{1}{C_2} + \cdots + \frac{1}{C_n}}$$



실습2. 병렬 커패시터

다음의 병렬 커패시터 회로를 만들고 커패시턴스를 측정해봅시다.

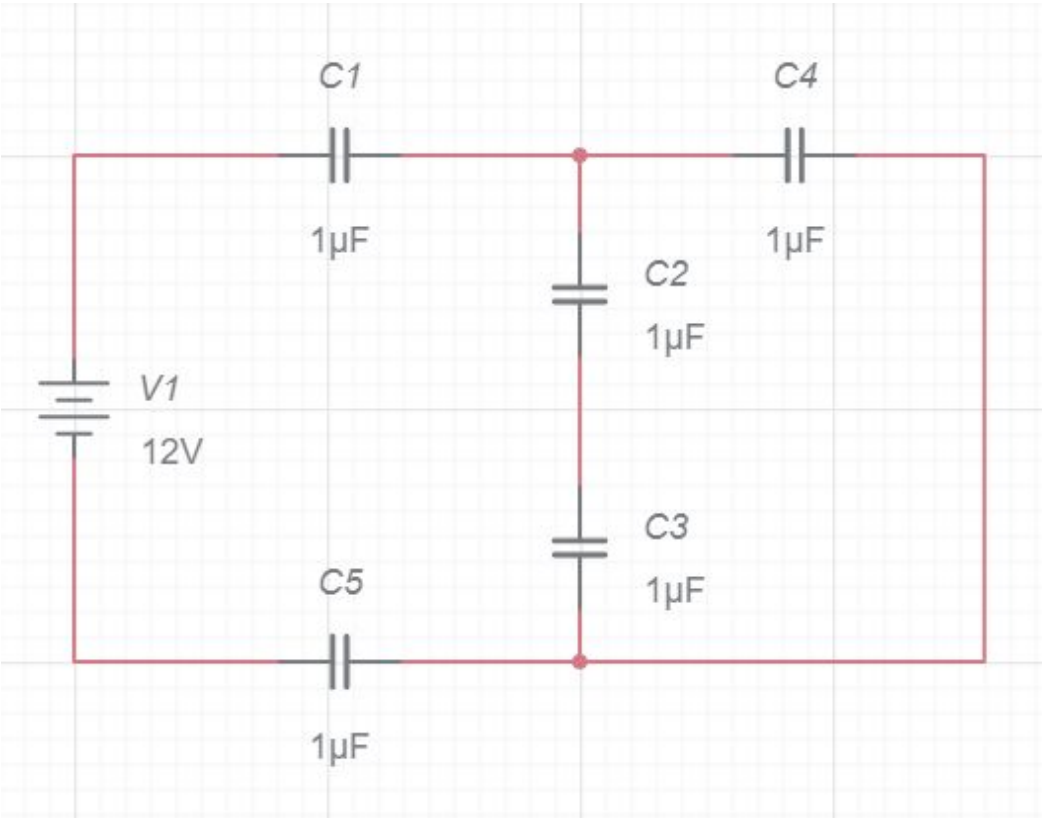


$$C_{EQ} = C_1 + C_2 + \cdots + C_n$$

병렬 커패시터의 총 커패시턴스

실습3. 직병렬 커패시터

다음의 직병렬 커패시터 회로를 만들고 커패시턴스를 측정해봅시다.



$$C_{EQ} = \frac{1}{\frac{1}{C_1} + \left(\frac{1}{C_4 + \frac{1}{\frac{1}{C_2 + C_3}}}\right) + \frac{1}{C_5}}$$